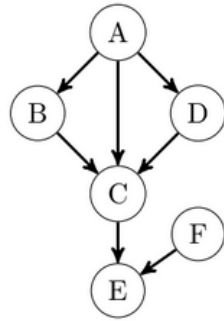


DMU Recap

DMU Recap

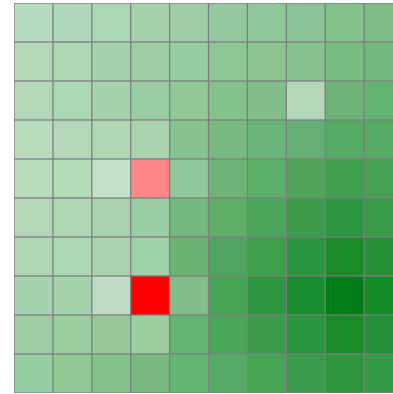
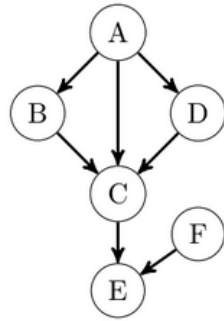
Probabilistic
Models



DMU Recap

Probabilistic
Models

Markov Decision
Processes

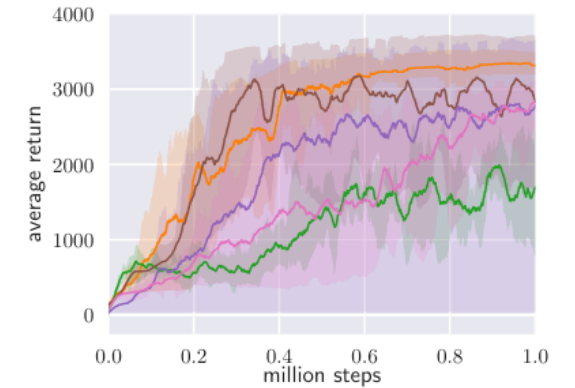
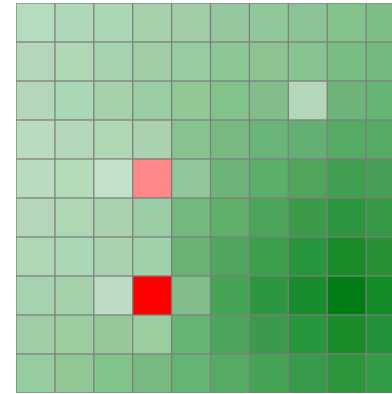
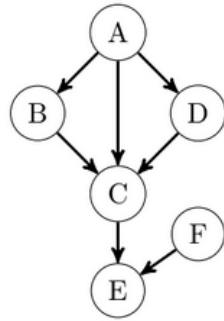


DMU Recap

Probabilistic
Models

Markov Decision
Processes

Reinforcement
Learning



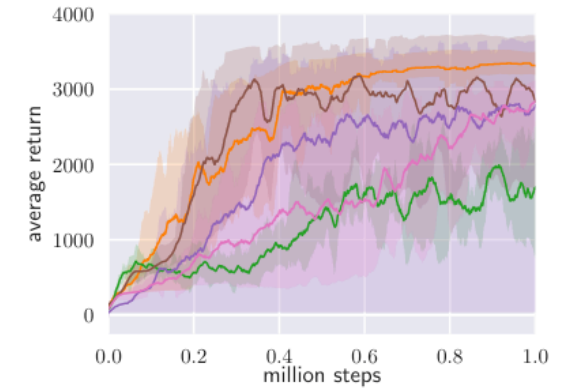
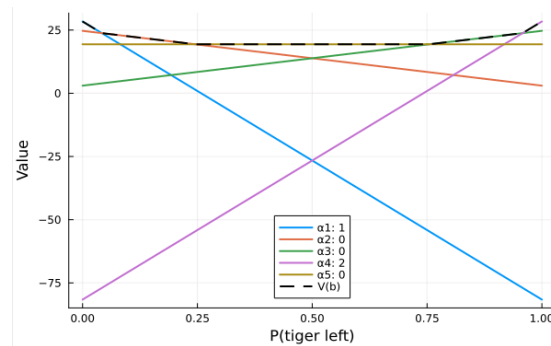
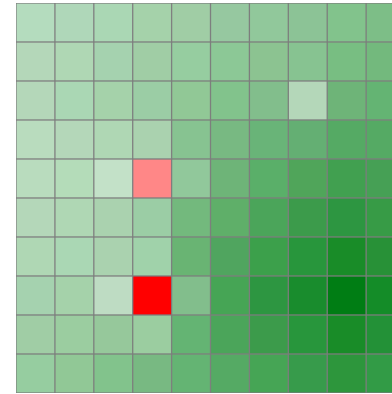
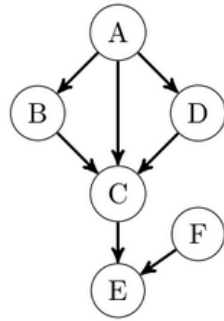
DMU Recap

Probabilistic
Models

Markov Decision
Processes

Reinforcement
Learning

POMDPs



DMU Recap

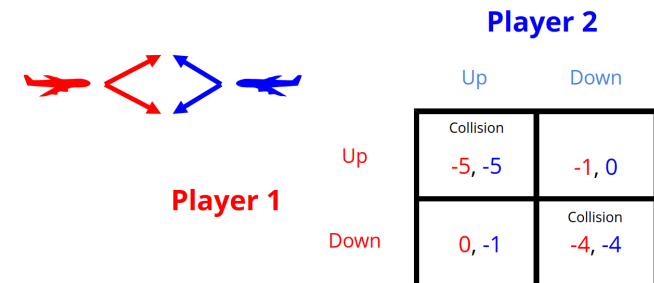
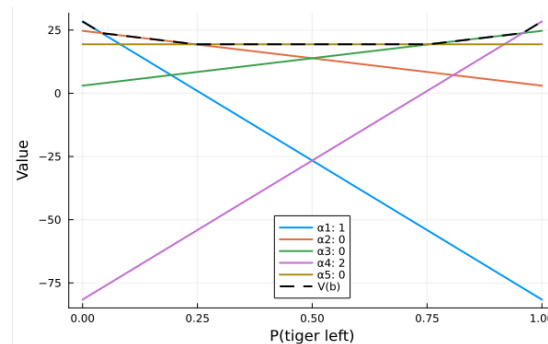
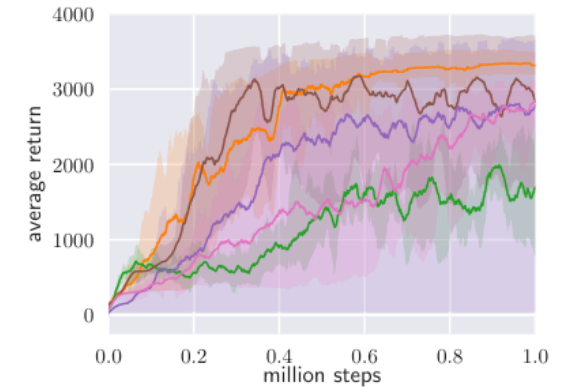
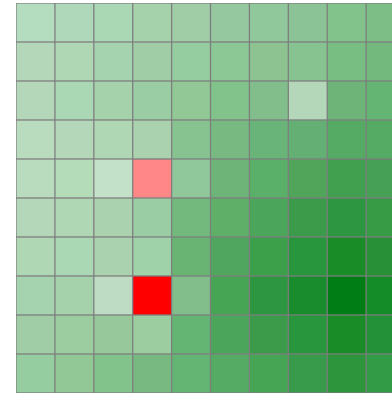
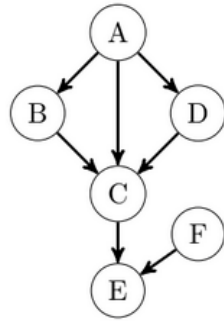
Probabilistic
Models

Markov Decision
Processes

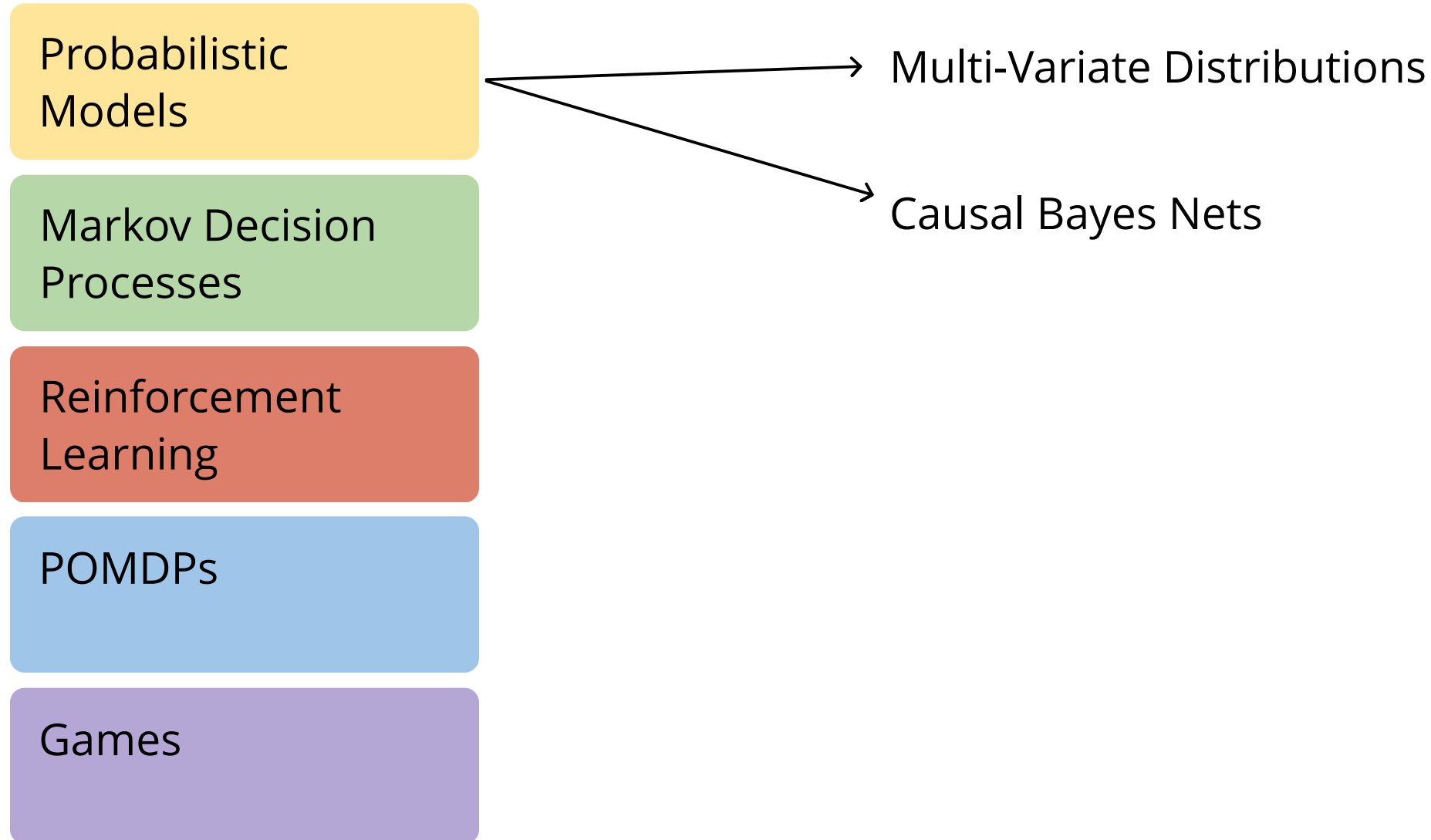
Reinforcement
Learning

POMDPs

Games



DMU Recap



Multi-Variate Distributions

Marginal: $P(A)$

Joint: $P(A, B)$

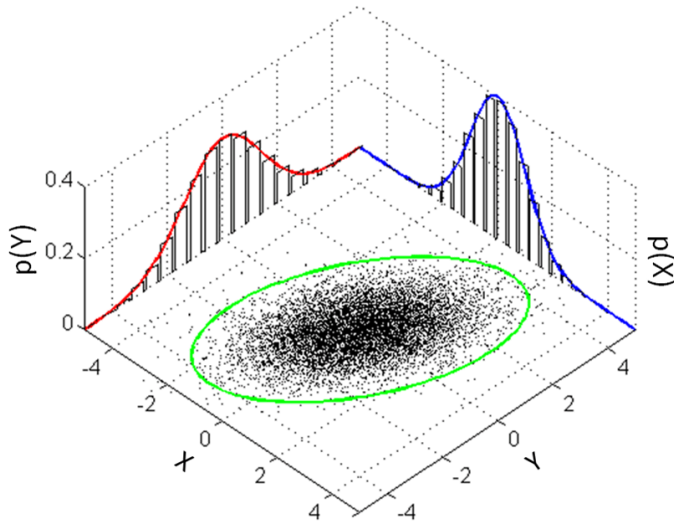
Conditional: $P(A \mid B)$

Multi-Variate Distributions

Marginal: $P(A)$

Joint: $P(A, B)$

Conditional: $P(A \mid B)$



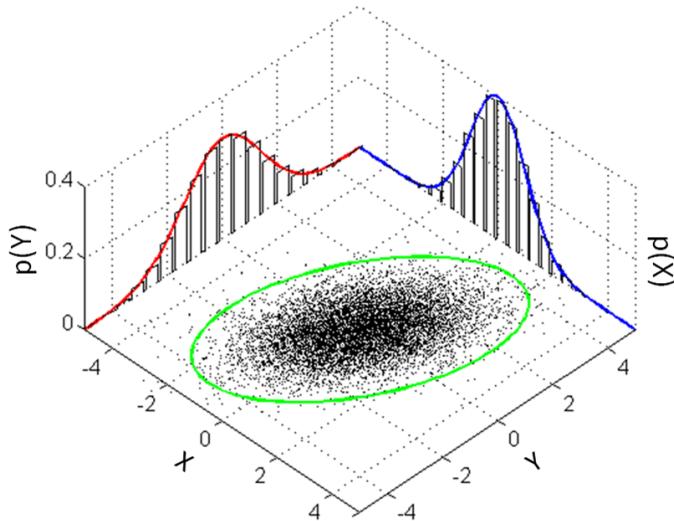
Multi-Variate Distributions

Marginal: $P(A)$

Joint: $P(A, B)$

Conditional: $P(A \mid B)$

3 Rules



Multi-Variate Distributions

Marginal: $P(A)$

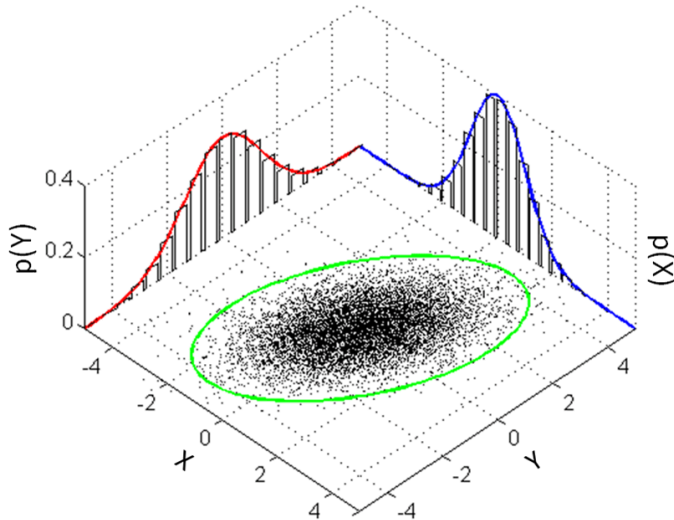
Joint: $P(A, B)$

Conditional: $P(A \mid B)$

3 Rules

$$1. 0 \leq P(X \mid Y) \leq 1$$

$$\sum_{x \in X} P(x \mid Y) = 1$$



Multi-Variate Distributions

Marginal: $P(A)$

Joint: $P(A, B)$

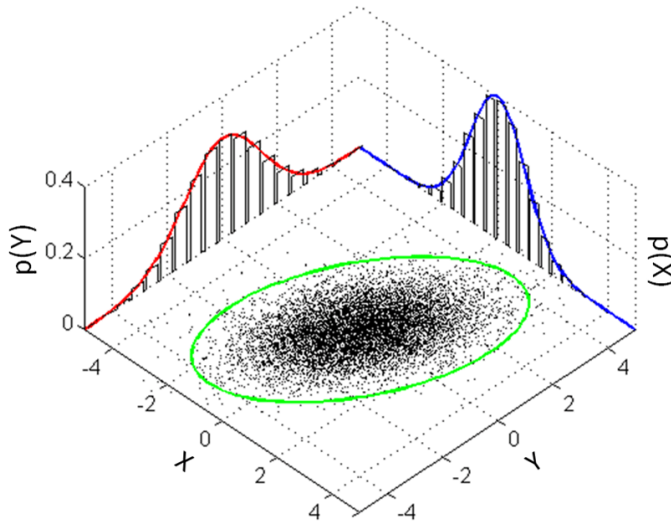
Conditional: $P(A \mid B)$

3 Rules

$$1. 0 \leq P(X \mid Y) \leq 1$$

$$\sum_{x \in X} P(x \mid Y) = 1$$

$$2. P(X) = \sum_{y \in Y} P(X, y)$$

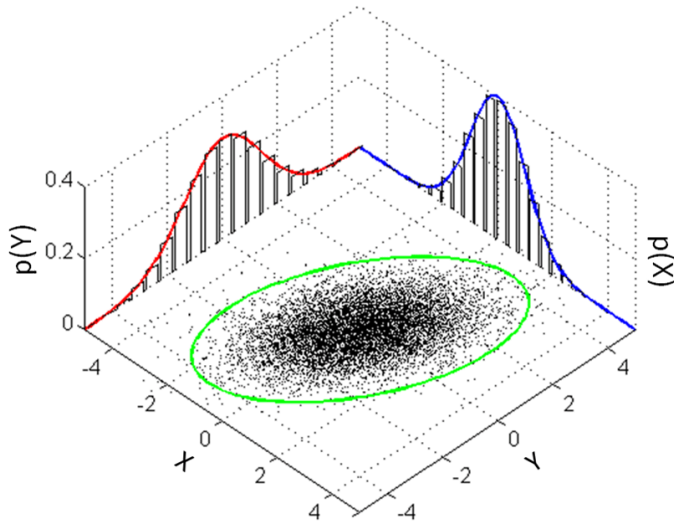


Multi-Variate Distributions

Marginal: $P(A)$

Joint: $P(A, B)$

Conditional: $P(A \mid B)$



3 Rules

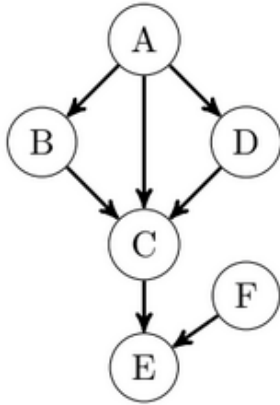
$$1. 0 \leq P(X \mid Y) \leq 1$$
$$\sum_{x \in X} P(x \mid Y) = 1$$

$$2. P(X) = \sum_{y \in Y} P(X, y)$$

$$3. P(X \mid Y) = \frac{P(X, Y)}{P(Y)}$$

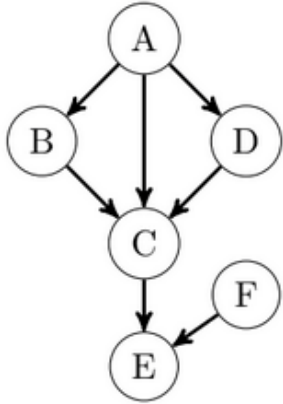
Causal Bayesian Networks

Causal Bayesian Networks



Each node is a **random variable** with a defined **distribution conditioned** on parents

Causal Bayesian Networks

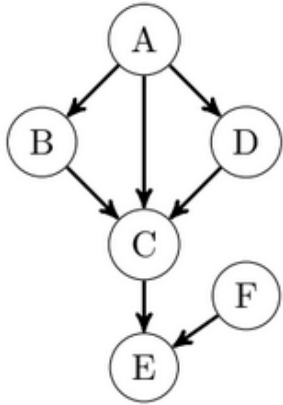


Each node is a **random variable** with a defined **distribution conditioned** on parents

Chain Rule

$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

Causal Bayesian Networks



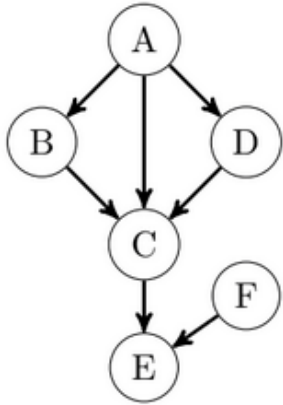
Each node is a **random variable** with a defined **distribution conditioned** on parents

Chain Rule

$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

(Naive) Inference

Causal Bayesian Networks



Each node is a **random variable** with a defined **distribution conditioned** on parents

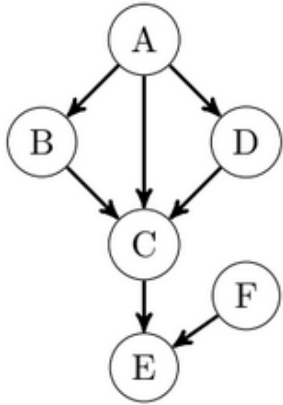
Chain Rule

$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

(Naive) Inference

$$P(Q = q \mid E = e) = \frac{P(Q=q, E=e)}{P(E=e)}$$

Causal Bayesian Networks



Each node is a **random variable** with a defined **distribution conditioned** on parents

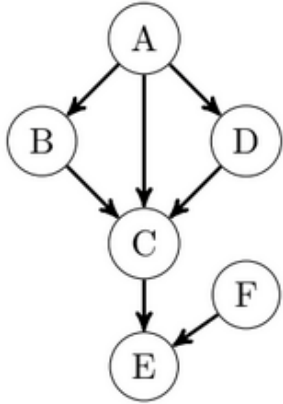
Chain Rule

$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

(Naive) Inference

$$P(Q = q \mid E = e) = \frac{P(Q=q, E=e)}{P(E=e)} = \frac{\sum_i P(Q=q, I=i, E=e)}{\sum_{i,q} P(Q=q, I=i, E=e)}$$

Causal Bayesian Networks



Each node is a **random variable** with a defined **distribution conditioned** on parents

Chain Rule

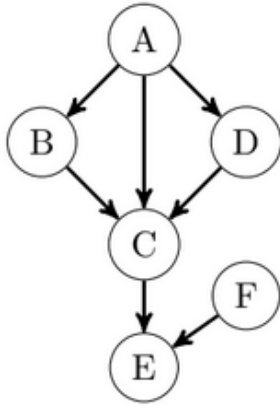
$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

(Naive) Inference

$$P(Q = q \mid E = e) = \frac{P(Q=q, E=e)}{P(E=e)} = \frac{\sum_i P(Q=q, I=i, E=e)}{\sum_{i,q} P(Q=q, I=i, E=e)}$$

Apply chain rule based on structure

Causal Bayesian Networks



Each node is a **random variable** with a defined **distribution conditioned** on parents

Chain Rule

$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

(Naive) Inference

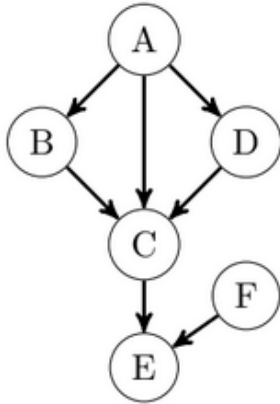
$$P(Q = q \mid E = e) = \frac{P(Q=q, E=e)}{P(E=e)} = \frac{\sum_i P(Q=q, I=i, E=e)}{\sum_{i,q} P(Q=q, I=i, E=e)}$$

Apply chain rule based on structure

Special Case: Bayes Rule

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Causal Bayesian Networks



Each node is a **random variable** with a defined **distribution conditioned** on parents

Chain Rule

$$P(X_{1:n}) = \prod_i P(X_i \mid Pa(X_i))$$

(Naive) Inference

$$P(Q = q \mid E = e) = \frac{P(Q=q, E=e)}{P(E=e)} = \frac{\sum_i P(Q=q, I=i, E=e)}{\sum_{i,q} P(Q=q, I=i, E=e)}$$

Apply chain rule based on structure

Special Case: Bayes Rule

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Conditional Independence

$X \perp Y \mid \mathcal{C}$ if all paths between X and Y are d-separated by $\mathcal{C}$

DMU Recap

Probabilistic
Models

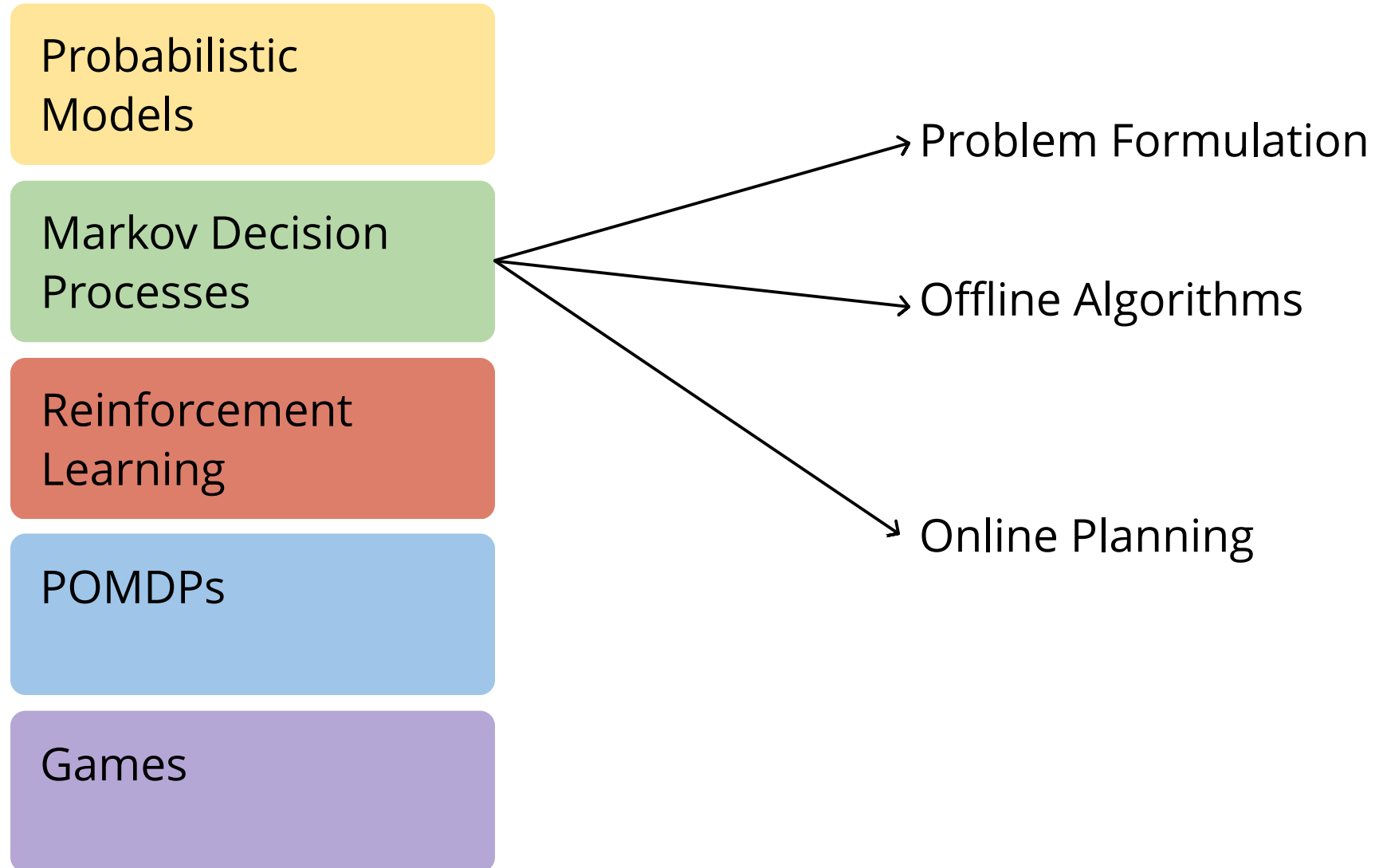
Markov Decision
Processes

Reinforcement
Learning

POMDPs

Games

DMU Recap



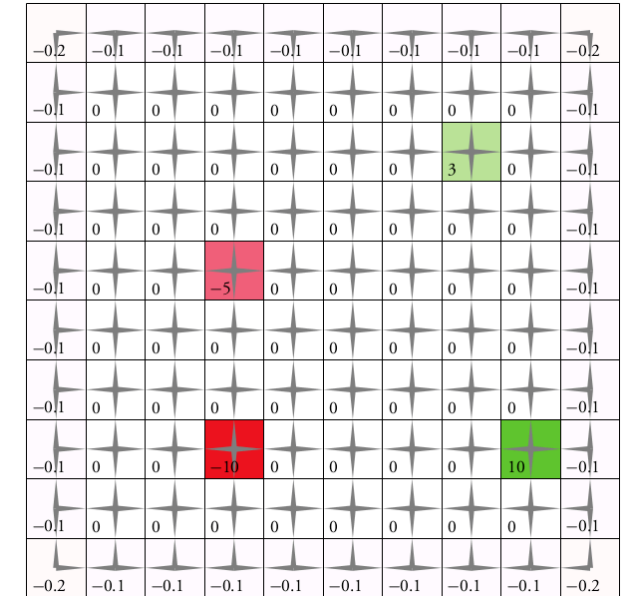
Markov Decision Processes

Markov Decision Processes

$$(S, A, R, T, \gamma)$$

Markov Decision Processes

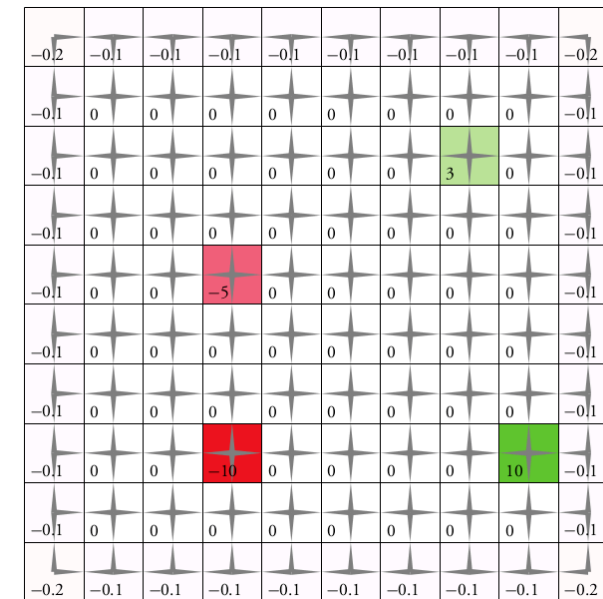
$$(S, A, R, T, \gamma)$$



Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$(S, A, R, T, \gamma)$$

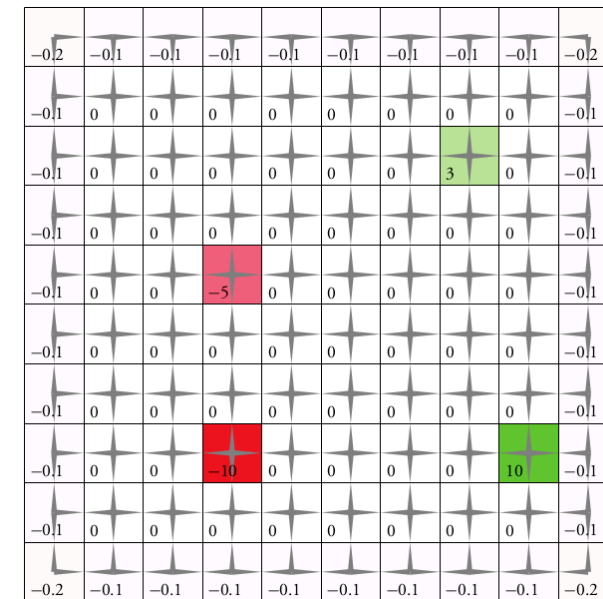


Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$(S, A, R, T, \gamma)$$



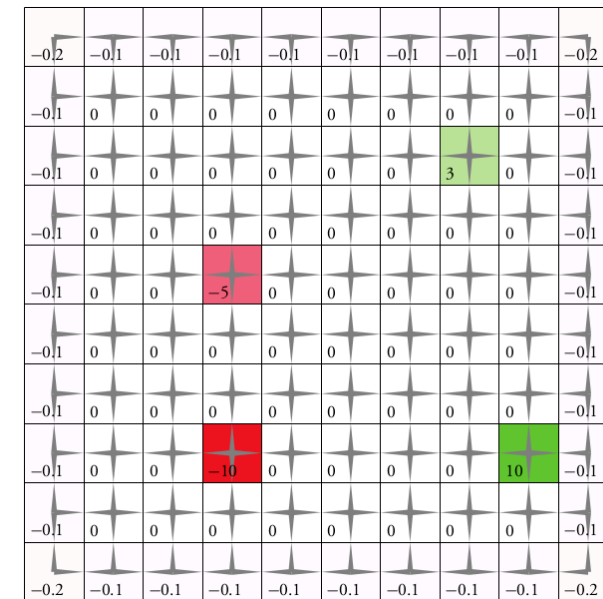
Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$(S, A, R, T, \gamma)$$

$$\underset{\pi}{\text{maximize}} E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$



Markov Decision Processes

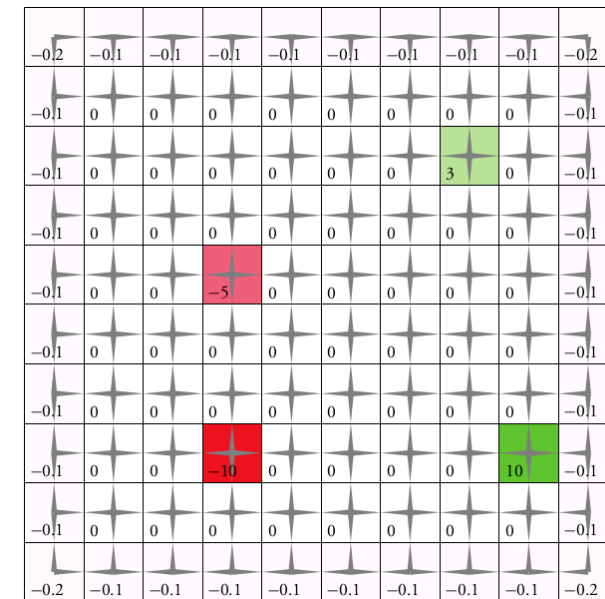
Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$Q^\pi(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a, a_t = \pi(s_t) \right]$$

$$(S, A, R, T, \gamma)$$

$$\underset{\pi}{\text{maximize}} E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$



Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

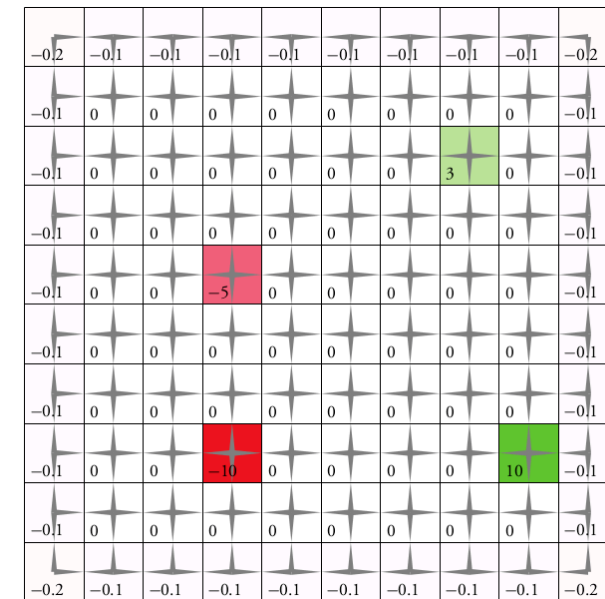
$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$Q^\pi(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a, a_t = \pi(s_t) \right]$$

$$V^\pi(s) = Q^\pi(s, \pi(s))$$

$$(S, A, R, T, \gamma)$$

$$\underset{\pi}{\text{maximize}} E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$



Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$Q^\pi(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a, a_t = \pi(s_t) \right]$$

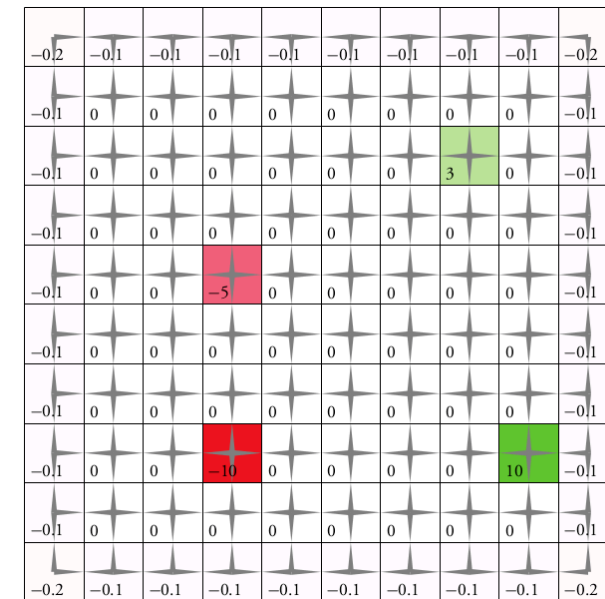
$$V^\pi(s) = Q^\pi(s, \pi(s))$$

$$V^\pi(s) = R(s, a) + \gamma E[V^\pi(s')]$$

Policy Evaluation

$$(S, A, R, T, \gamma)$$

$$\text{maximize}_{\pi} E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$



Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$Q^\pi(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a, a_t = \pi(s_t) \right]$$

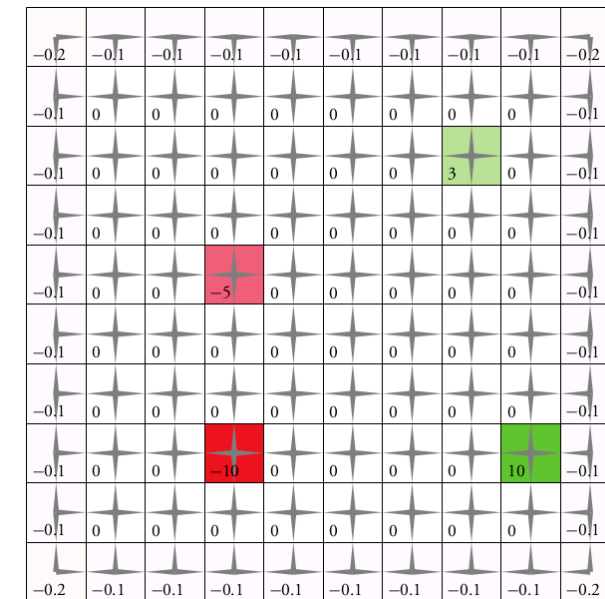
$$V^\pi(s) = Q^\pi(s, \pi(s))$$

$$V^\pi(s) = R(s, a) + \gamma E[V^\pi(s')]$$

$$V^*(s) = \max_a \{R(s, a) + \gamma E[V^*(s')]\}$$

$$(S, A, R, T, \gamma)$$

$$\underset{\pi}{\text{maximize}} E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$



Policy Evaluation

Bellman's Equation: Certificate of Optimality

Markov Decision Processes

Examples: $S = \{1, 2, 3\}$ or $S = \mathbb{R}^2$

$$s = (x, \dot{x}) \in S = \mathbb{R}^2$$

$$Q^\pi(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a, a_t = \pi(s_t) \right]$$

$$V^\pi(s) = Q^\pi(s, \pi(s))$$

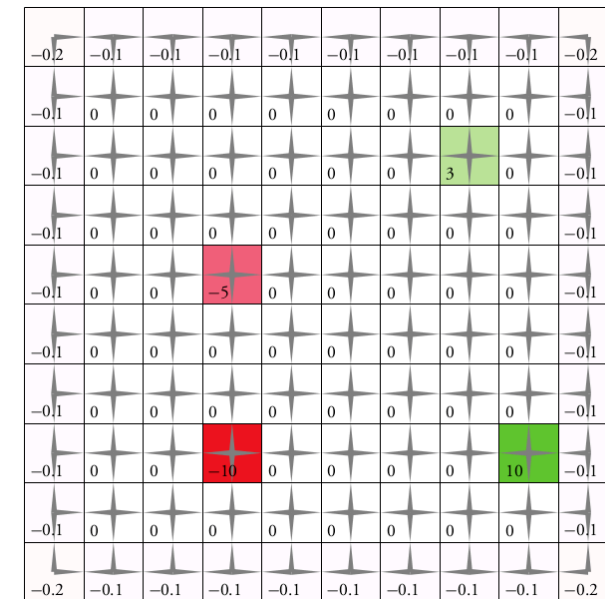
$$V^\pi(s) = R(s, a) + \gamma E[V^\pi(s')]$$

$$V^*(s) = \max_a \{R(s, a) + \gamma E[V^*(s')]\}$$

$$B[V](s) = \max_a \{R(s, a) + \gamma E[V(s')]\}$$

$$(S, A, R, T, \gamma)$$

$$\underset{\pi}{\text{maximize}} E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$



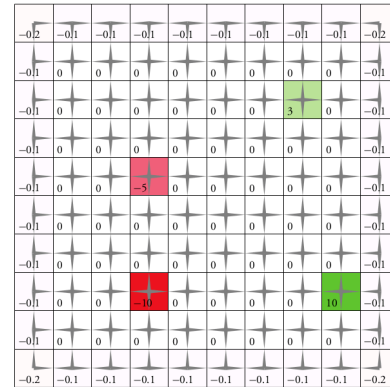
Policy Evaluation

Bellman's Equation: Certificate of Optimality

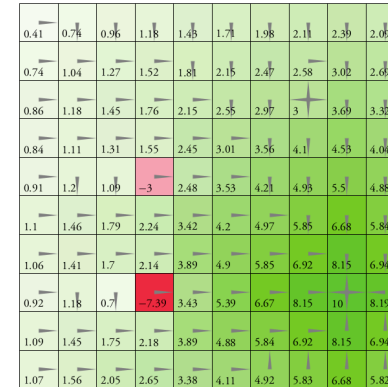
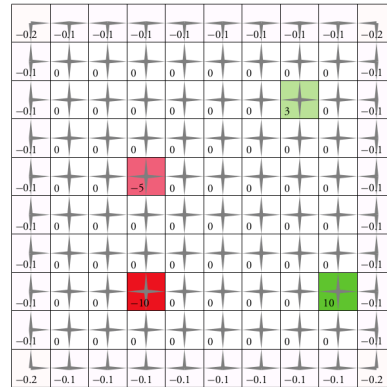
Bellman's Operator

Offline MDP Algorithms

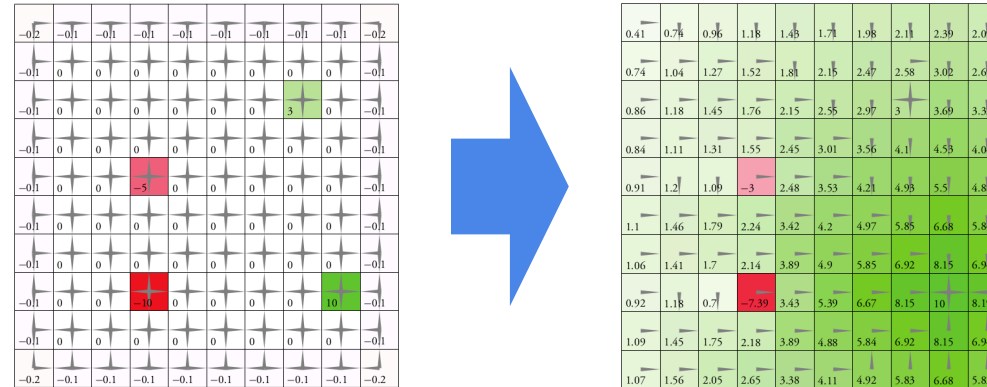
Offline MDP Algorithms



Offline MDP Algorithms



Offline MDP Algorithms



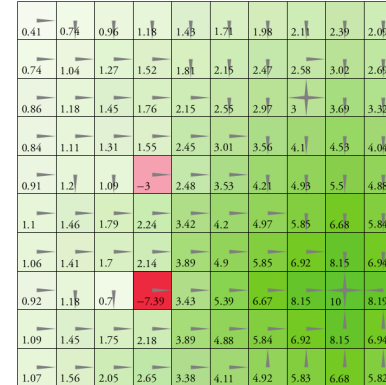
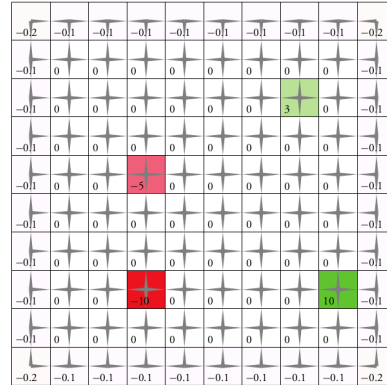
Policy Iteration

loop

Evaluate Policy

Improve Policy

Offline MDP Algorithms



Policy Iteration

loop

Evaluate Policy

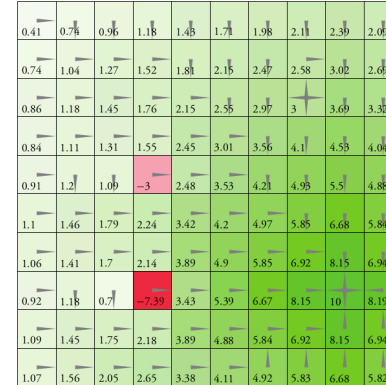
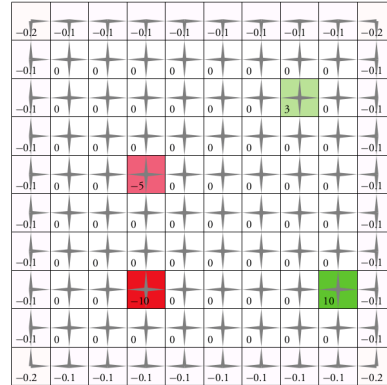
Improve Policy

Value Iteration

loop

$$V \leftarrow B[V]$$

Offline MDP Algorithms



Policy Iteration

loop

Evaluate Policy

Improve Policy

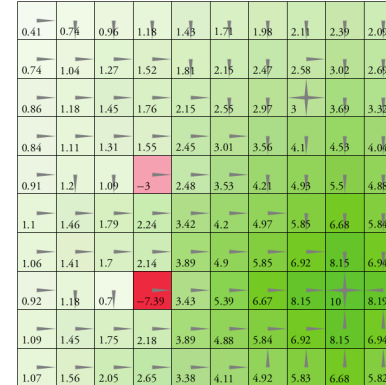
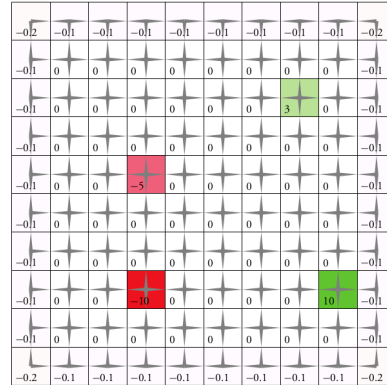
Finds optimal policy because
policy always improves and there
are a finite number of policies

Value Iteration

loop

$$V \leftarrow B[V]$$

Offline MDP Algorithms



Policy Iteration

loop

Evaluate Policy

Improve Policy

Finds optimal policy because
policy always improves and there
are a finite number of policies

Value Iteration

loop

$$V \leftarrow B[V]$$

Converges because B is
a contraction mapping

Online MDP Planning

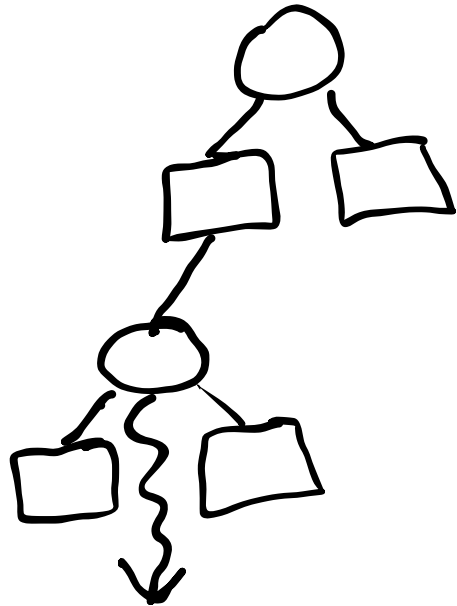
Online MDP Planning

Monte Carlo Tree Search

Online MDP Planning

Monte Carlo Tree Search

Search
Expand
Rollout
Backup

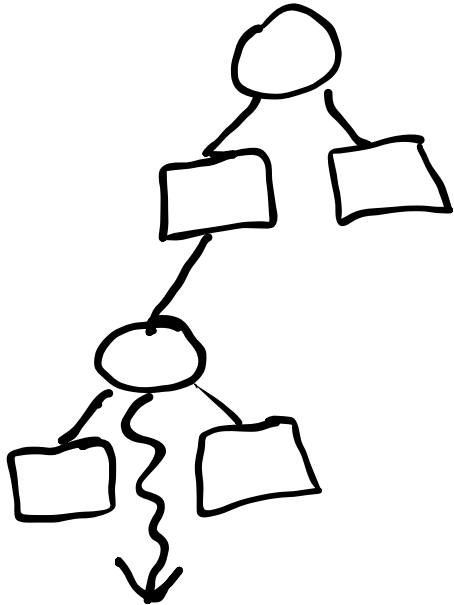


Online MDP Planning

Monte Carlo Tree Search

Search
Expand
Rollout
Backup

$$Q(s,a) + c \sqrt{\frac{\log N(s)}{N(s,a)}}$$

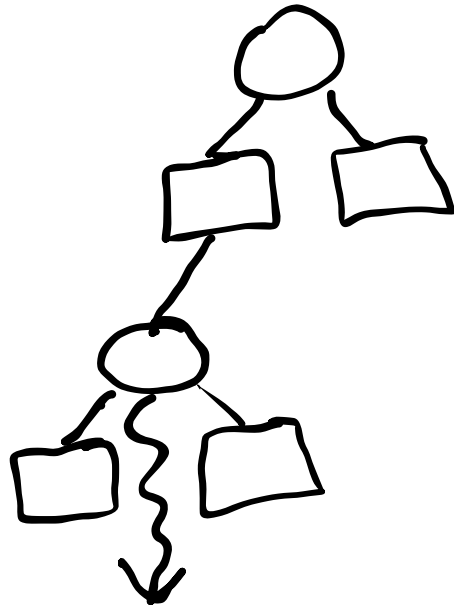


Online MDP Planning

Monte Carlo Tree Search

Search
Expand
Rollout
Backup

$$Q(s,a) + c \sqrt{\frac{\log N(s)}{N(s,a)}}$$



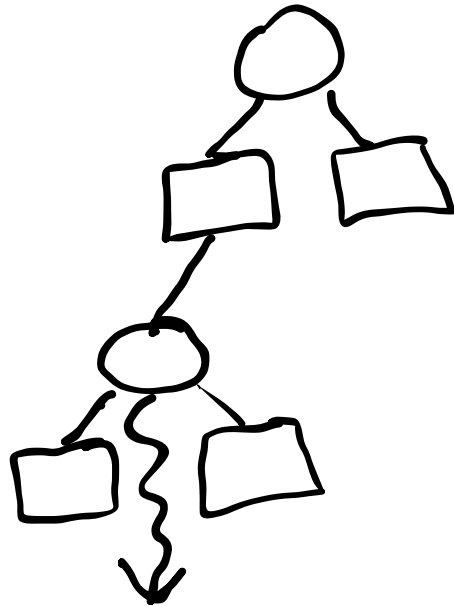
Sparse Sampling

Online MDP Planning

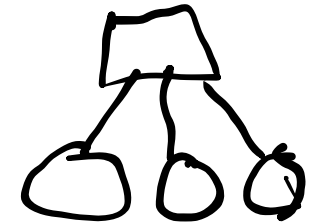
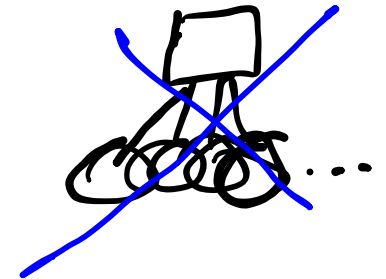
Monte Carlo Tree Search

Search
Expand
Rollout
Backup

$$Q(s,a) + c \sqrt{\frac{\log N(s)}{N(s,a)}}$$



Sparse Sampling

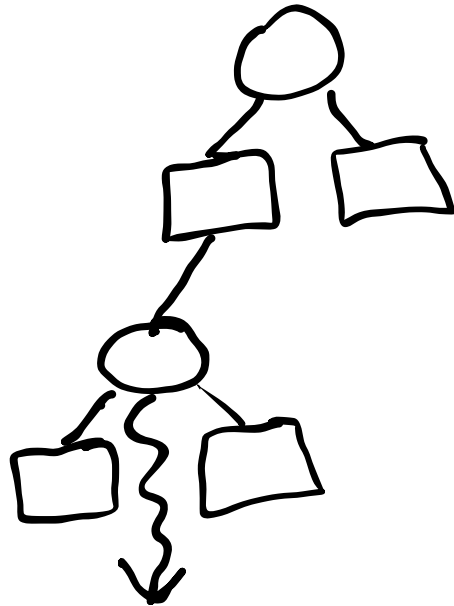


Online MDP Planning

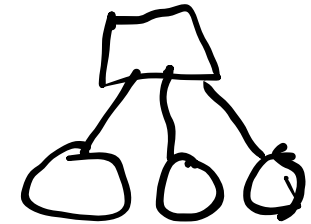
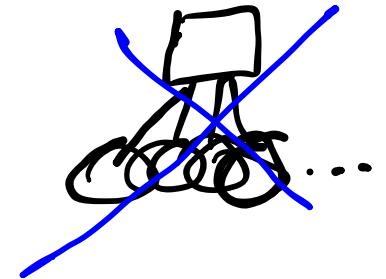
Monte Carlo Tree Search

Search
Expand
Rollout
Backup

$$Q(s,a) + c \sqrt{\frac{\log N(s)}{N(s,a)}}$$



Sparse Sampling



Guarantees *independent* of $|S|!!$

DMU Recap

Probabilistic
Models

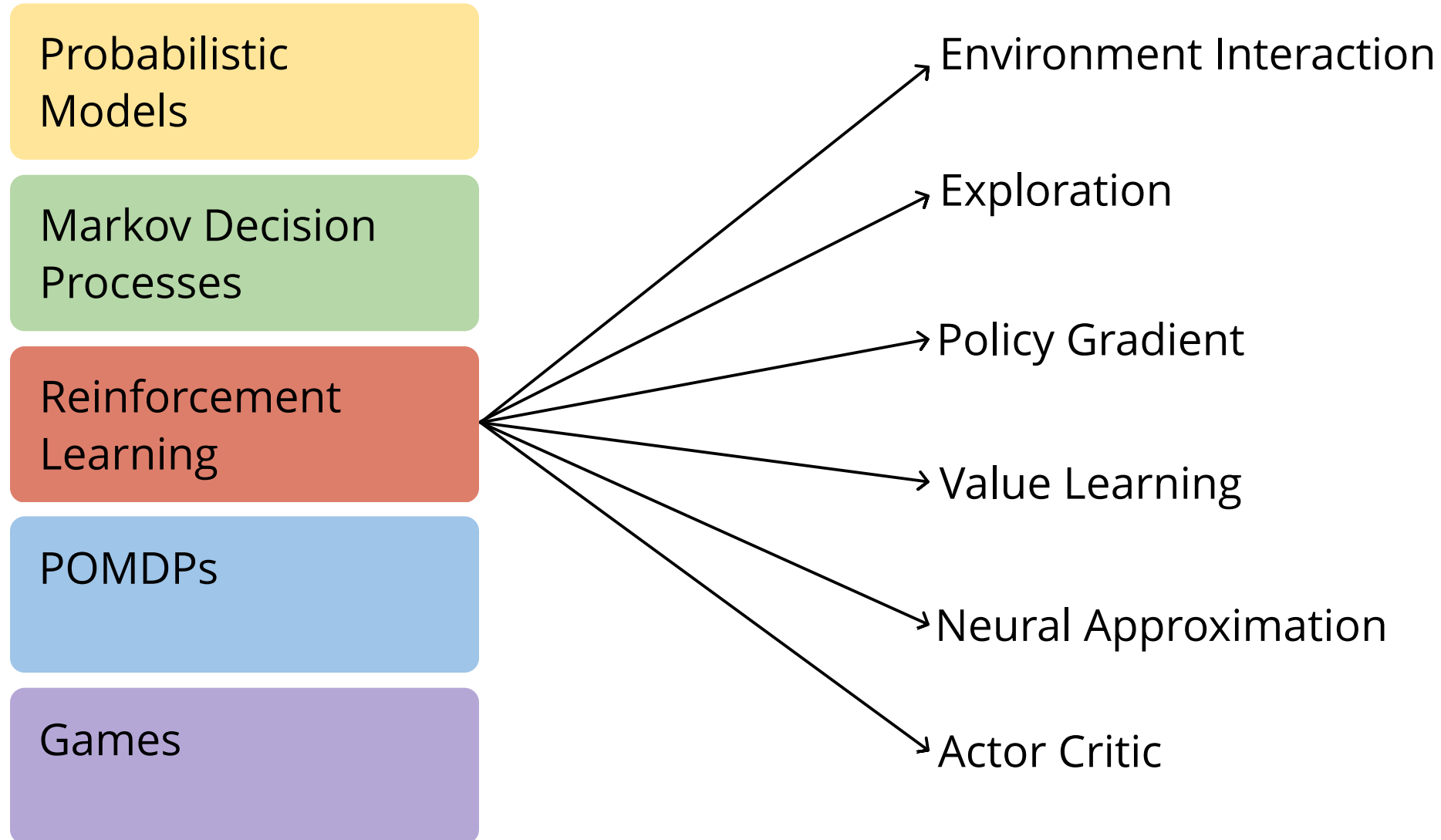
Markov Decision
Processes

Reinforcement
Learning

POMDPs

Games

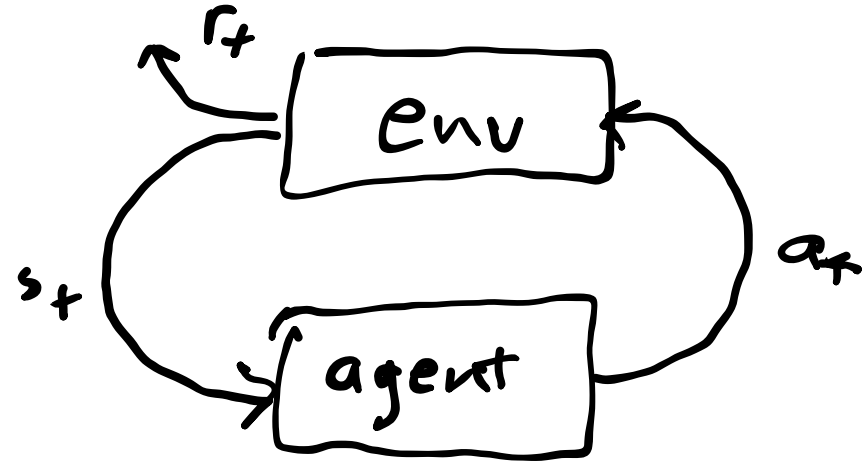
DMU Recap



Reinforcement Learning

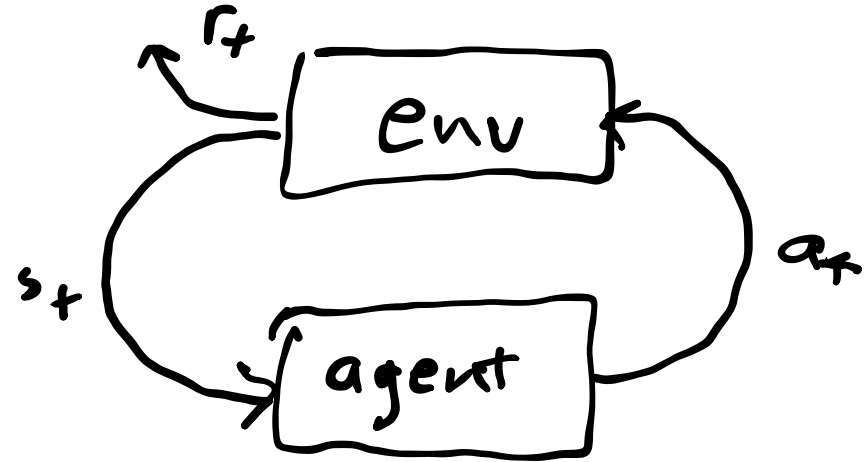
Challenges:

Reinforcement Learning



Challenges:

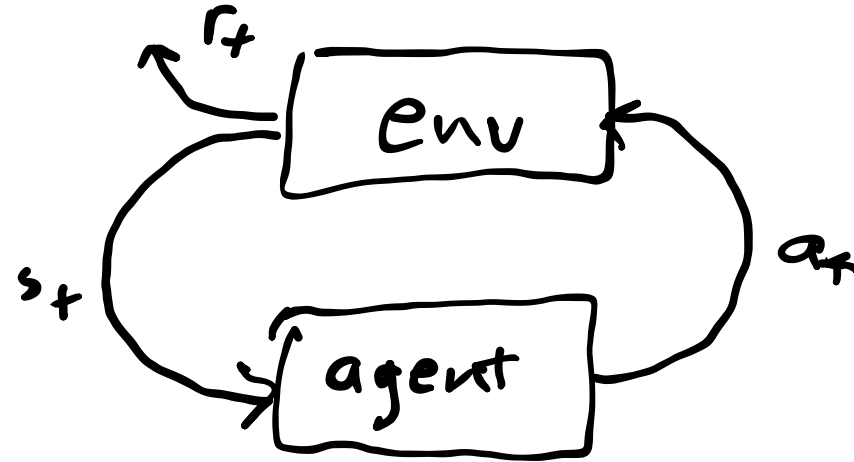
Reinforcement Learning



Challenges:

1. Exploration and Exploitation

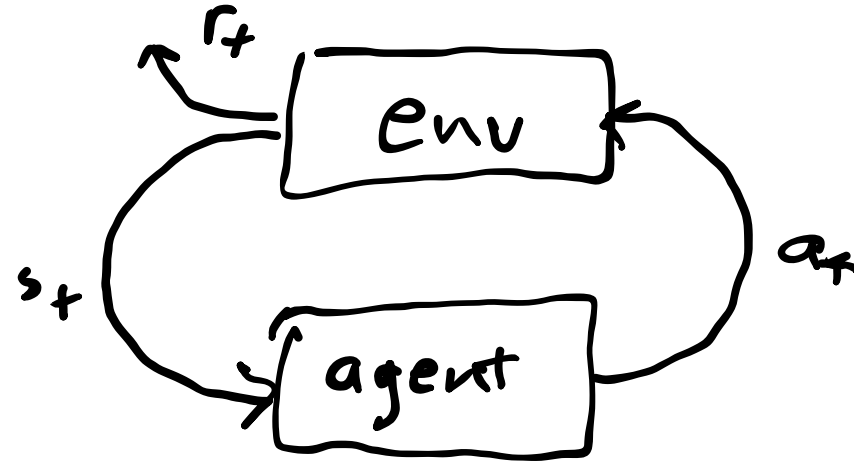
Reinforcement Learning



Challenges:

1. Exploration and Exploitation
2. Credit Assignment

Reinforcement Learning



Challenges:

1. Exploration and Exploitation
2. Credit Assignment
3. Generalization

Exploration

Exploration

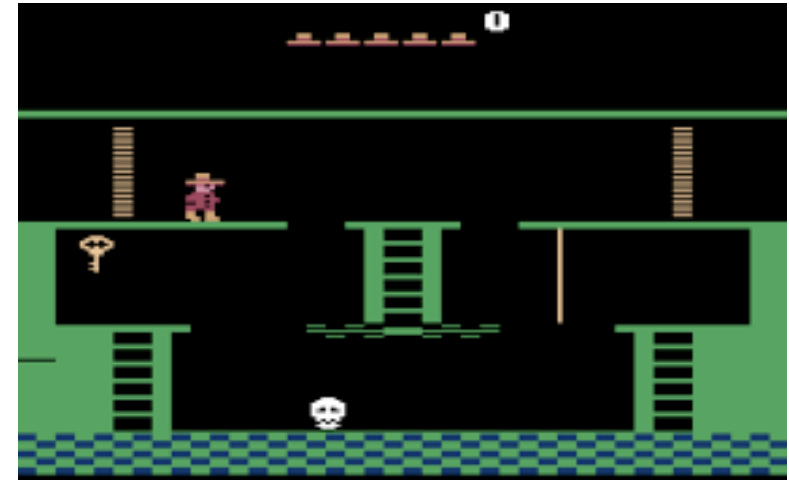
Bandits

- ϵ -greedy
- softmax
- UCB
- Thompson Sampling
- Optimal DP Solution (solving a POMDP!)

Exploration

Bandits

- ϵ -greedy
- softmax
- UCB
- Thompson Sampling
- Optimal DP Solution (solving a POMDP!)



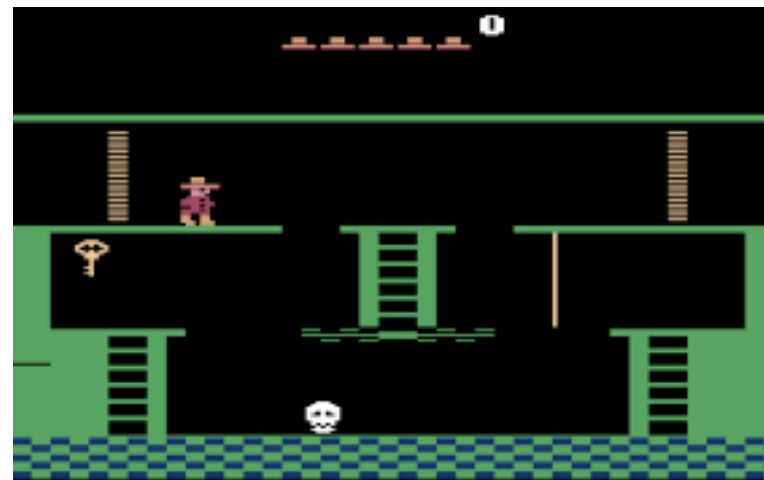
Montezuma's Revenge!

Exploration

Bandits

- ϵ -greedy
- softmax
- UCB
- Thompson Sampling
- Optimal DP Solution (solving a POMDP!)

- Pseudocounts

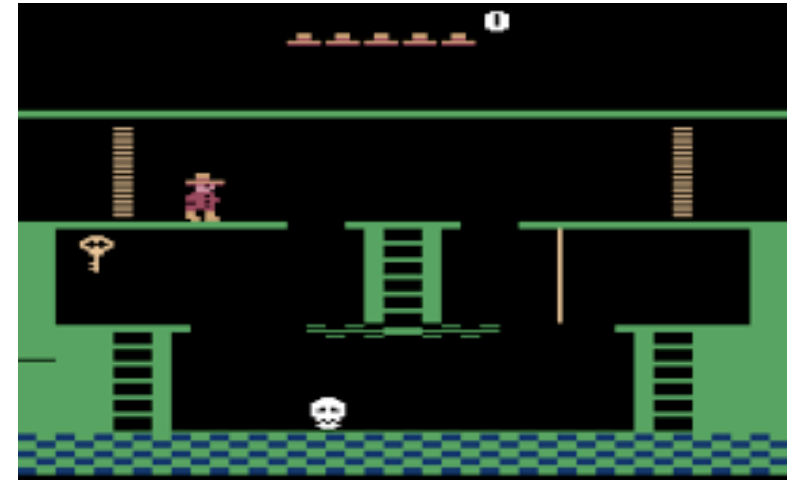


Montezuma's Revenge!

Exploration

Bandits

- ϵ -greedy
- softmax
- UCB
- Thompson Sampling
- Optimal DP Solution (solving a POMDP!)



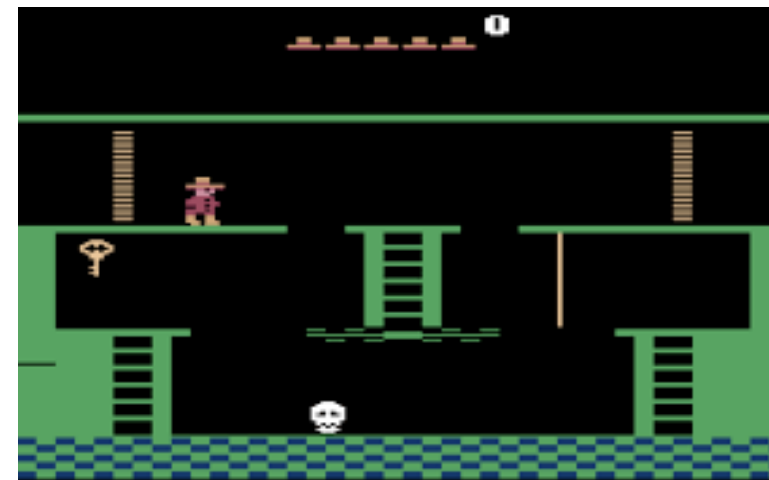
Montezuma's Revenge!

- Pseudocounts
- Curiosity: extra reward for bad prediction

Exploration

Bandits

- ϵ -greedy
- softmax
- UCB
- Thompson Sampling
- Optimal DP Solution (solving a POMDP!)



Montezuma's Revenge!

- Pseudocounts
- Curiosity: extra reward for bad prediction
- Random network distillation

RL Algorithms

RL Algorithms

Model
Based

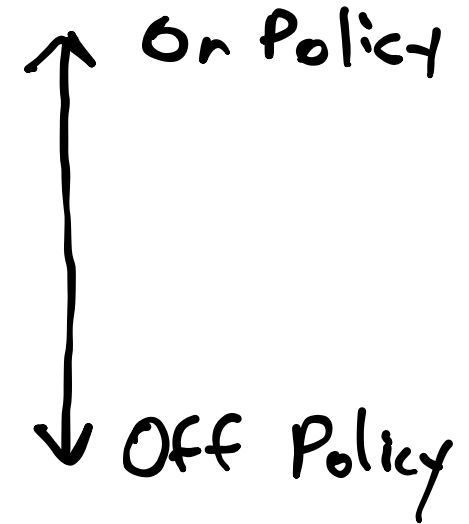
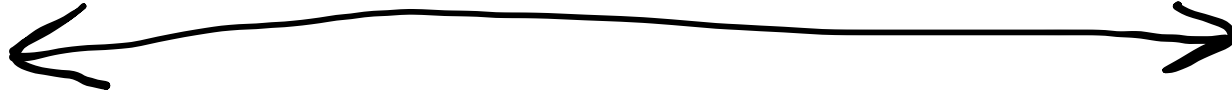
Model
Free



RL Algorithms

Model
Based

Model
Free



RL Algorithms

Model
Based

Model
Free



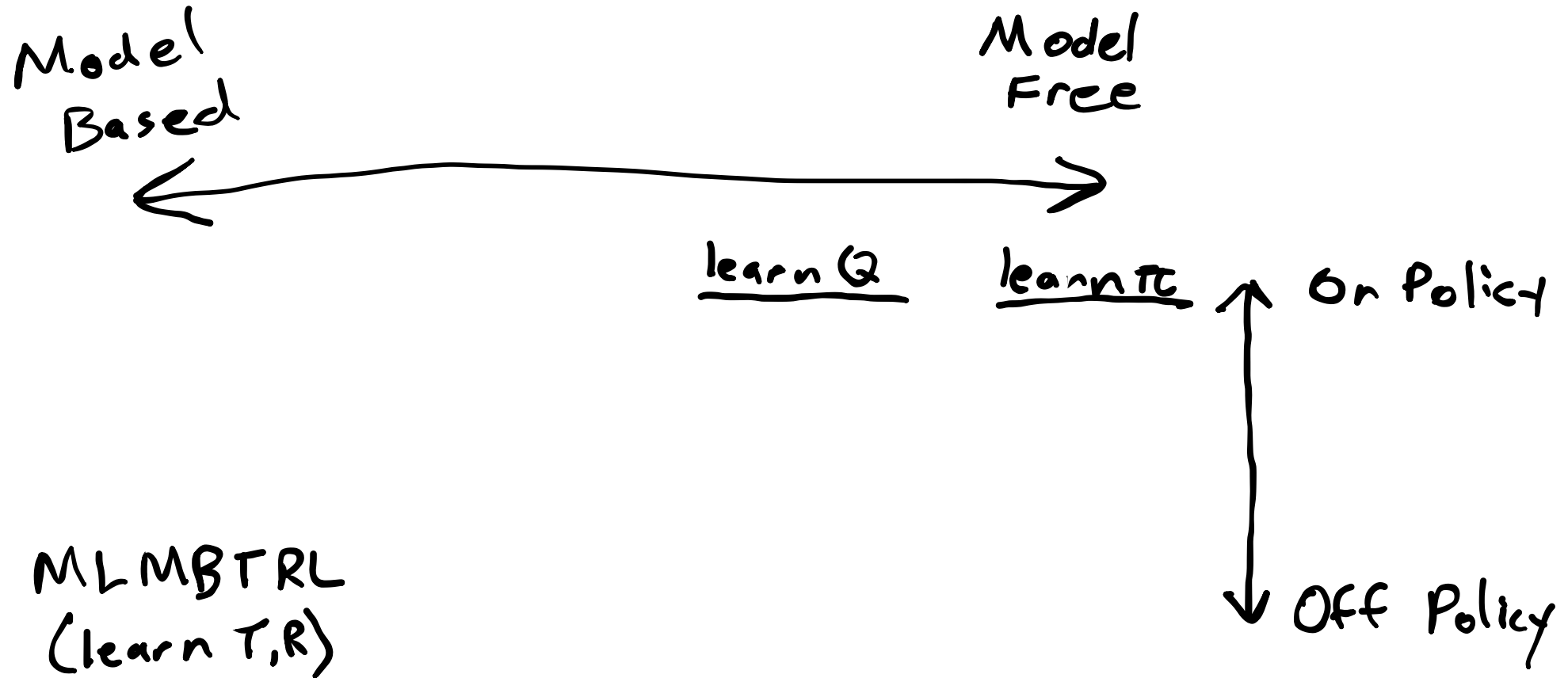
MLMBTRL
(learn T, R)

On Policy

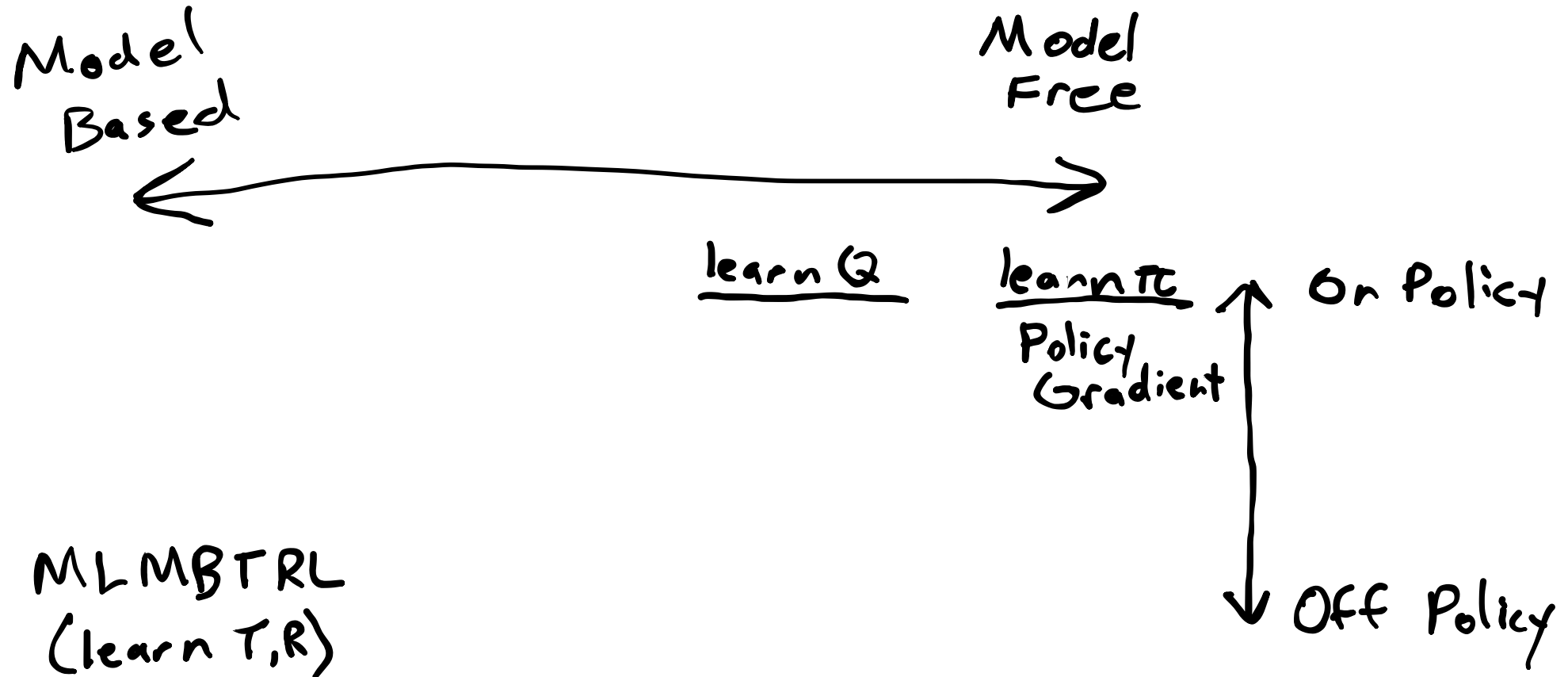
Off Policy



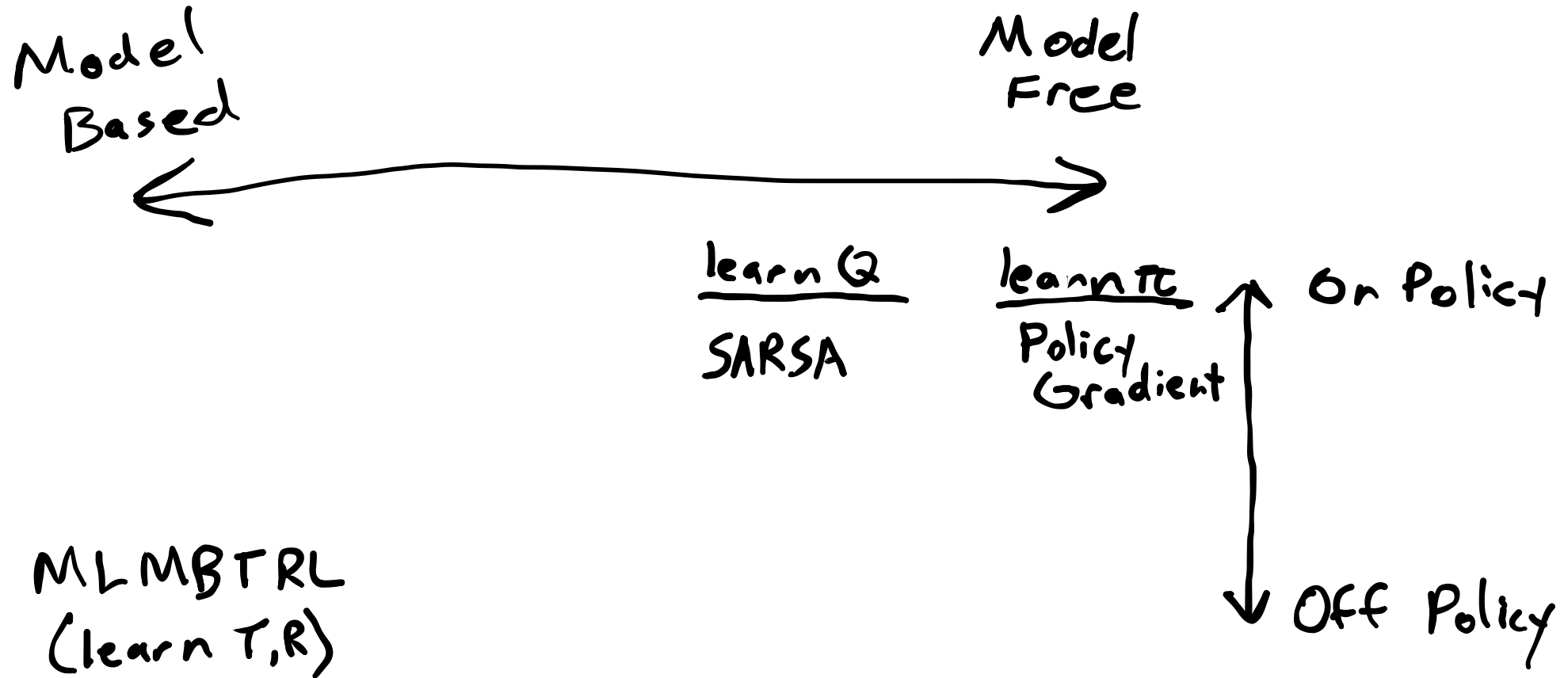
RL Algorithms



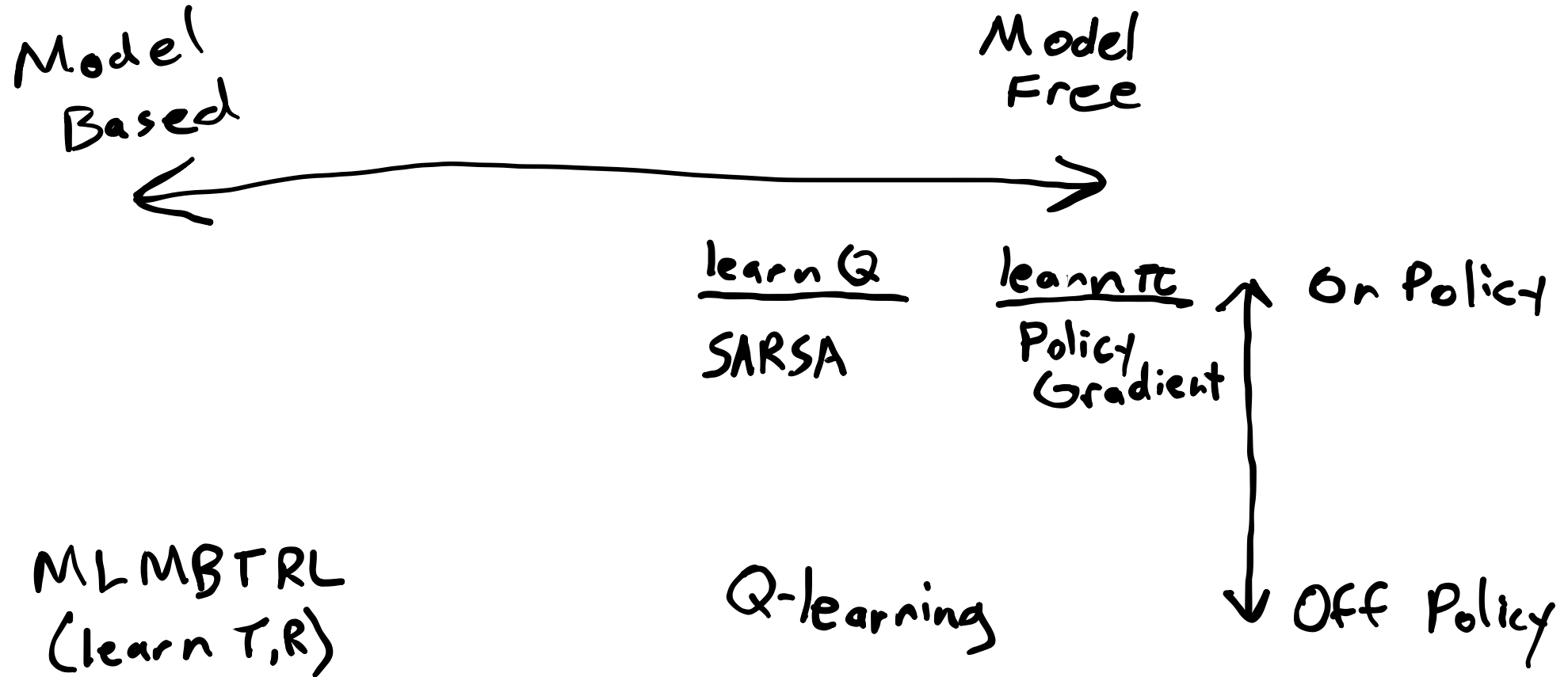
RL Algorithms



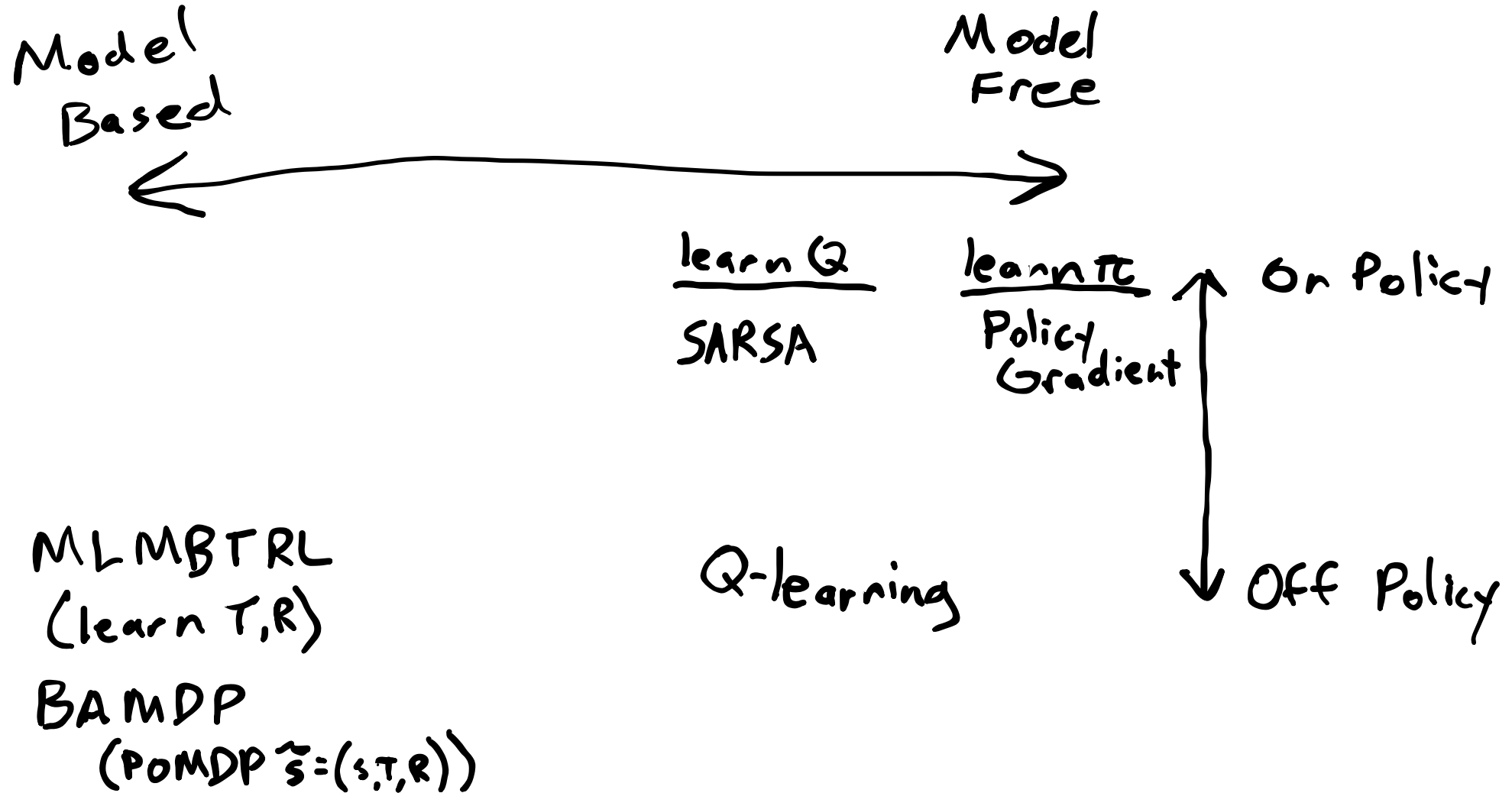
RL Algorithms



RL Algorithms



RL Algorithms



Policy Gradient

Policy Gradient

- Likelihood ratio trick $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$

Policy Gradient

- Likelihood ratio trick
- Causality

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

Policy Gradient

- Likelihood ratio trick
- Causality
- Baseline Subtraction

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

Policy Gradient

- Likelihood ratio trick
- Causality
- Baseline Subtraction

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

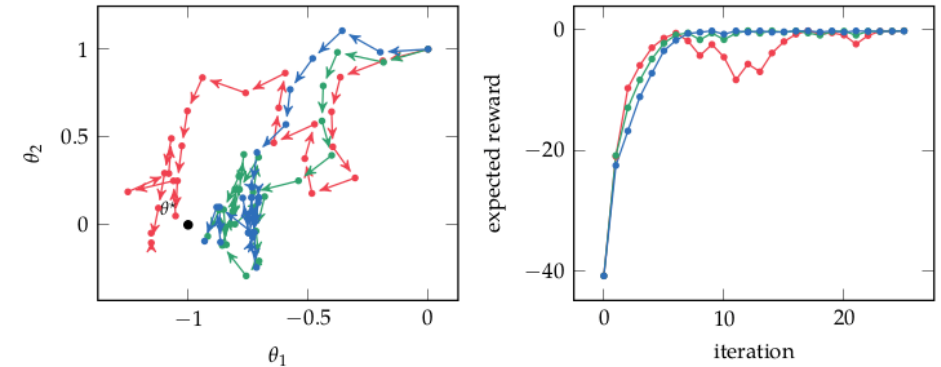
$$\nabla U(\theta) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\theta} \log \pi_{\theta}(a^{(k)} | s^{(k)}) \gamma^{k-1} (r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)})) \right]$$

Policy Gradient

- Likelihood ratio trick
- Causality
- Baseline Subtraction

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

$$\nabla U(\theta) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\theta} \log \pi_{\theta}(a^{(k)} | s^{(k)}) \gamma^{k-1} (r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)})) \right]$$



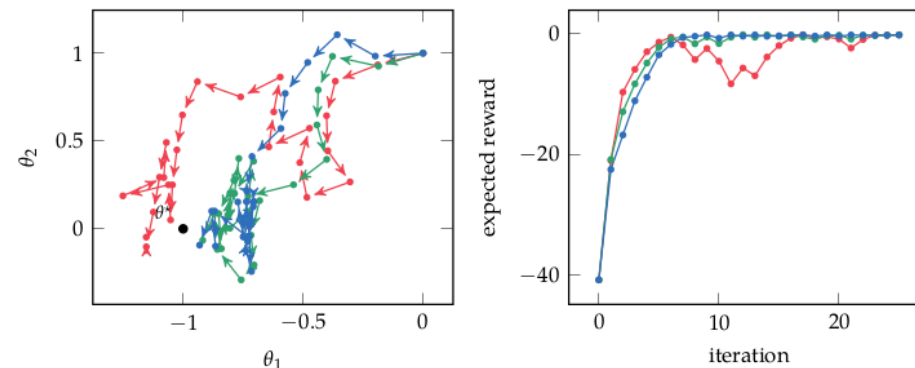
Policy Gradient

- Likelihood ratio trick
- Causality
- Baseline Subtraction

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

$$\nabla U(\theta) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\theta} \log \pi_{\theta}(a^{(k)} | s^{(k)}) \gamma^{k-1} \left(r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)}) \right) \right]$$

- Natural Gradient (TRPO)

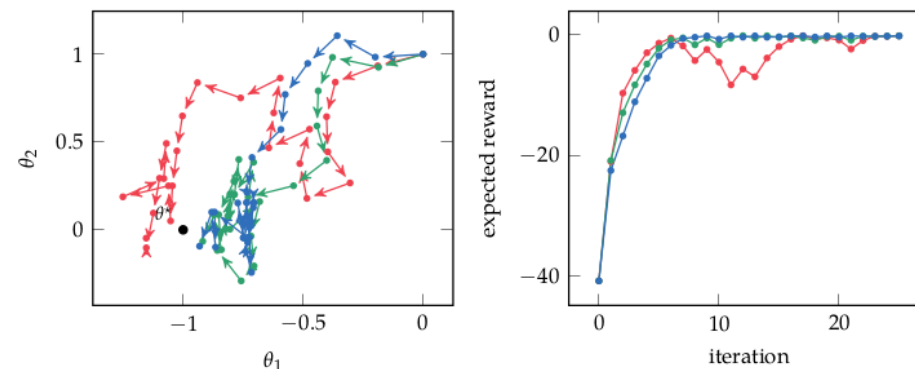


Policy Gradient

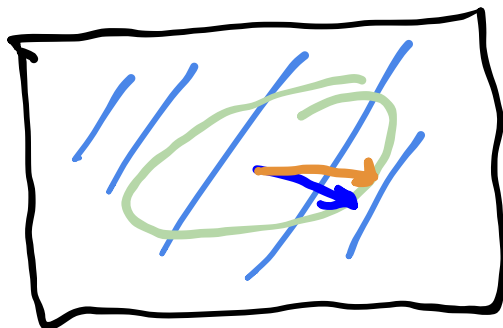
- Likelihood ratio trick
- Causality
- Baseline Subtraction

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

$$\nabla U(\theta) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\theta} \log \pi_{\theta}(a^{(k)} | s^{(k)}) \gamma^{k-1} (r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)})) \right]$$



- Natural Gradient (TRPO)



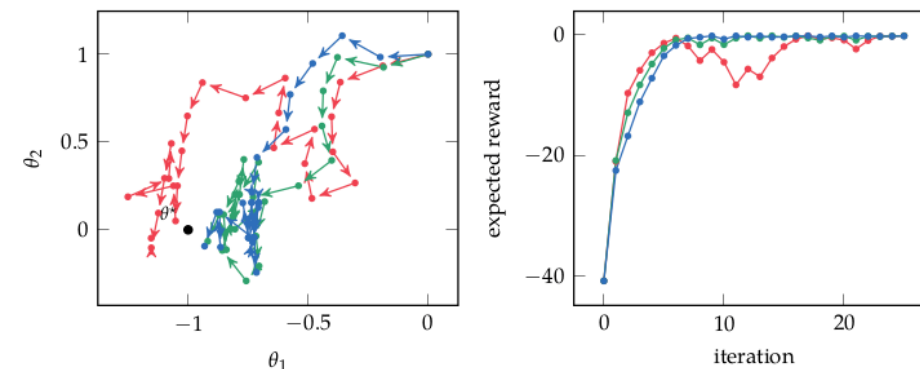
KL div.
Bound

Policy Gradient

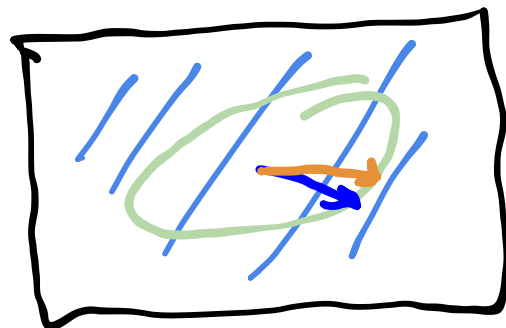
- Likelihood ratio trick
- Causality
- Baseline Subtraction

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

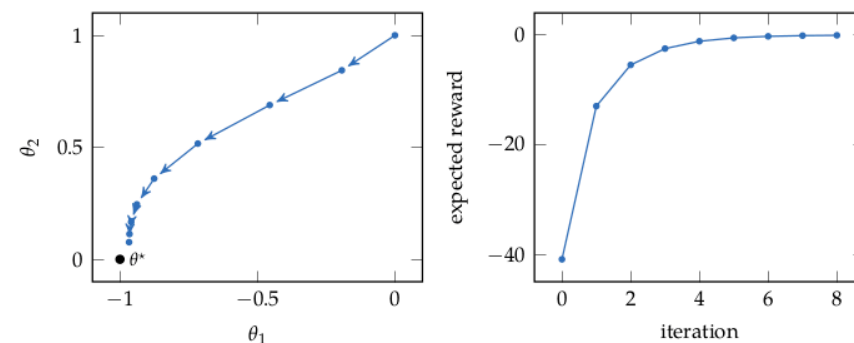
$$\nabla U(\theta) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\theta} \log \pi_{\theta}(a^{(k)} | s^{(k)}) \gamma^{k-1} (r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)})) \right]$$



- Natural Gradient (TRPO)



KL div.
Bound



Q-Learning

Q-Learning

SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r_t + \gamma Q(s', a') - Q(s, a))$$

Q-Learning

SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r_t + \gamma Q(s', a') - Q(s, a))$$

Eligibility Traces

Q-Learning

SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r_t + \gamma Q(s', a') - Q(s, a))$$

Eligibility Traces

Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r_t + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Q-Learning

SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r_t + \gamma Q(s', a') - Q(s, a))$$

Eligibility Traces

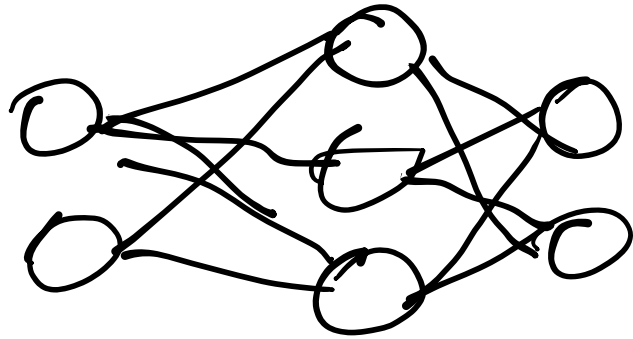
Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r_t + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

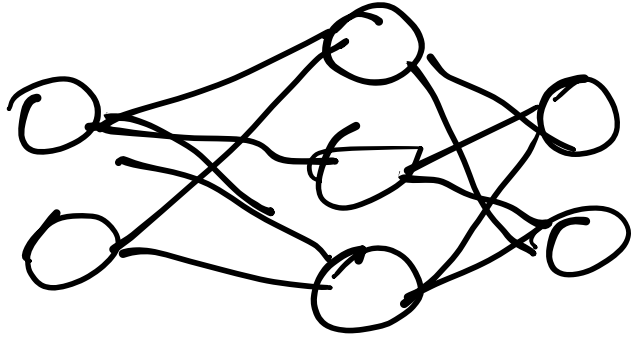
Double Q Learning

Neural Networks and DQN

Neural Networks and DQN

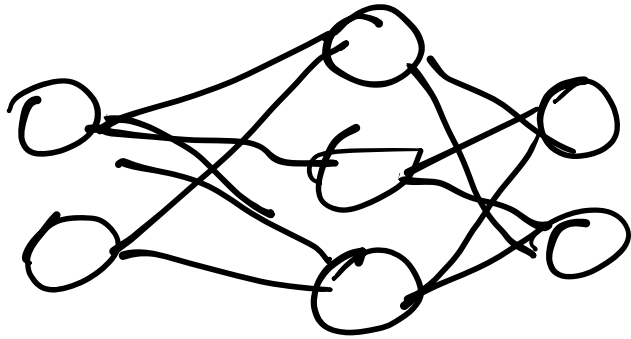


Neural Networks and DQN



$$f_{\theta}(x) = \sigma(W_2\sigma(W_1x + b_1) + b_2)$$

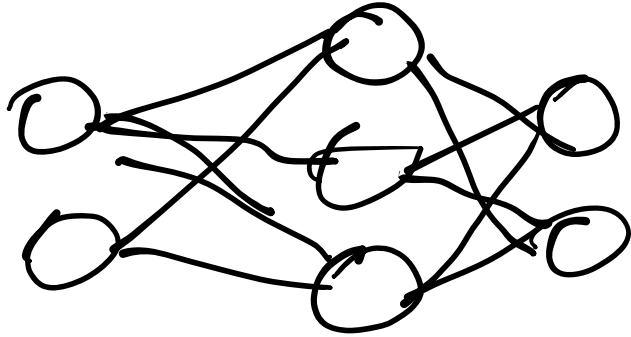
Neural Networks and DQN



$$f_{\theta}(x) = \sigma(W_2\sigma(W_1x + b_1) + b_2)$$

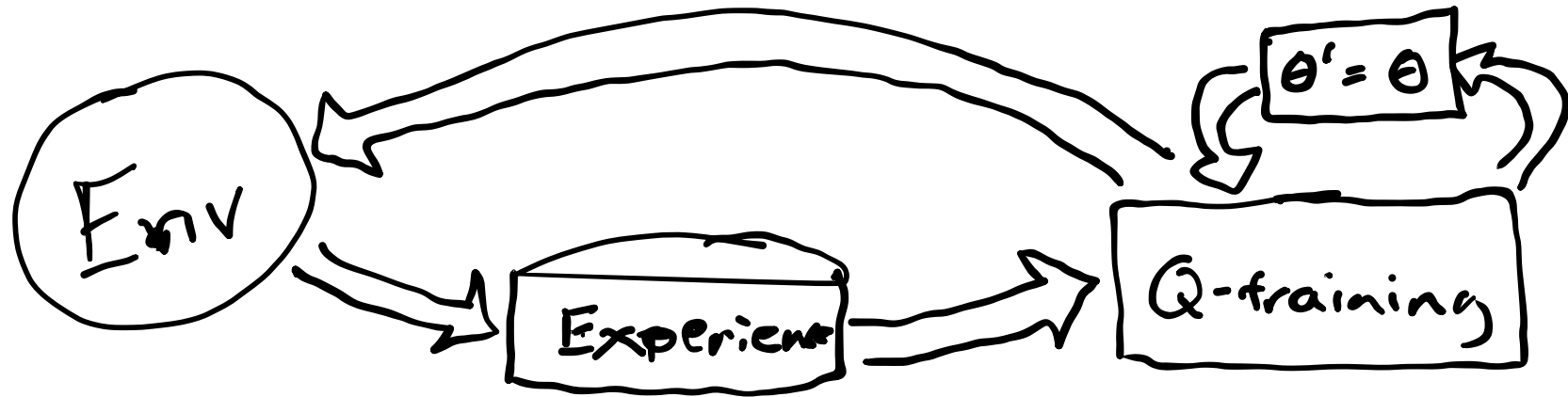
Backprop

Neural Networks and DQN



$$f_{\theta}(x) = \sigma(W_2\sigma(W_1x + b_1) + b_2)$$

Backprop



Actor-Critic

$$\nabla U(\boldsymbol{\theta}) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a^{(k)} \mid s^{(k)}) \gamma^{k-1} \left(r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)}) \right) \right]$$

Actor-Critic

$$\nabla U(\theta) = \mathbb{E}_{\tau} \left[\sum_{k=1}^d \nabla_{\theta} \log \pi_{\theta}(a^{(k)} \mid s^{(k)}) \gamma^{k-1} \left(r_{\text{to-go}}^{(k)} - r_{\text{base}}(s^{(k)}) \right) \right]$$

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) \gamma^k \hat{A}_k$$

Actor-Critic

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) \gamma^k \hat{A}_k$$

Actor-Critic

- Actor: π_θ
- Critic: Q_ϕ, V_ϕ, A_ϕ

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k \mid s_k) \gamma^k \hat{A}_k$$

Actor-Critic

- Actor: π_θ
- Critic: Q_ϕ, V_ϕ, A_ϕ

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k \hat{A}_k$$

$$A_{\text{GAE}}(s_k, a_k) \approx \sum_{t=k}^{\infty} (\gamma \lambda)^{t-k} (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))$$

Actor-Critic

- Actor: π_θ
- Critic: Q_ϕ, V_ϕ, A_ϕ

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k \hat{A}_k$$

$$A_{\text{GAE}}(s_k, a_k) \approx \sum_{t=k}^{\infty} (\gamma \lambda)^{t-k} (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))$$

Soft Actor Critic (SAC)

$$J(\pi) = E \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

Actor-Critic

- Actor: π_θ
- Critic: Q_ϕ, V_ϕ, A_ϕ

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k \hat{A}_k$$

$$A_{\text{GAE}}(s_k, a_k) \approx \sum_{t=k}^{\infty} (\gamma \lambda)^{t-k} (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))$$

Soft Actor Critic (SAC)

$$J(\pi) = E \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

Proximal Policy Optimization (PPO)

$$E_{\substack{s \sim b_{\gamma, \theta} \\ a \sim \pi_\theta(\cdot | s)}} \left[\min \left(\frac{\pi_{\theta'}(a | s)}{\pi_\theta(a | s)} A_\phi(s, a), \text{clamp} \left(\frac{\pi_{\theta'}(a | s)}{\pi_\theta(a | s)}, 1 - \epsilon, 1 + \epsilon \right) A_\phi(s, a) \right) \right]$$

Actor-Critic

- Actor: π_θ
- Critic: Q_ϕ, V_ϕ, A_ϕ

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k \hat{A}_k$$

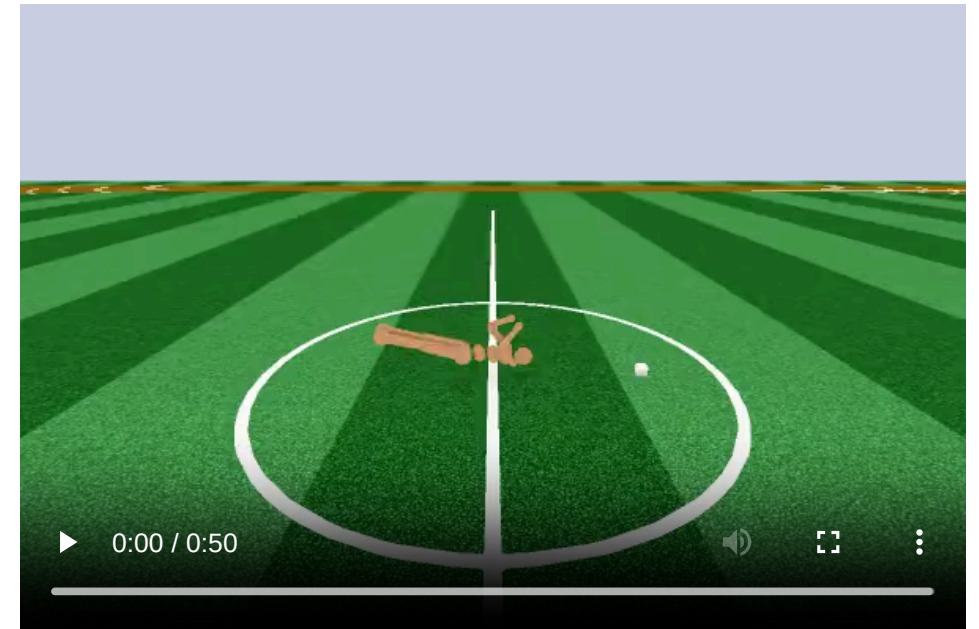
$$A_{\text{GAE}}(s_k, a_k) \approx \sum_{t=k}^{\infty} (\gamma \lambda)^{t-k} (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))$$

Soft Actor Critic (SAC)

$$J(\pi) = E \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

Proximal Policy Optimization (PPO)

$$E_{\substack{s \sim b_{\gamma, \theta} \\ a \sim \pi_\theta(\cdot | s)}} \left[\min \left(\frac{\pi_{\theta'}(a | s)}{\pi_\theta(a | s)} A_\phi(s, a), \text{clamp} \left(\frac{\pi_{\theta'}(a | s)}{\pi_\theta(a | s)}, 1 - \epsilon, 1 + \epsilon \right) A_\phi(s, a) \right) \right]$$



DMU Recap

Probabilistic
Models

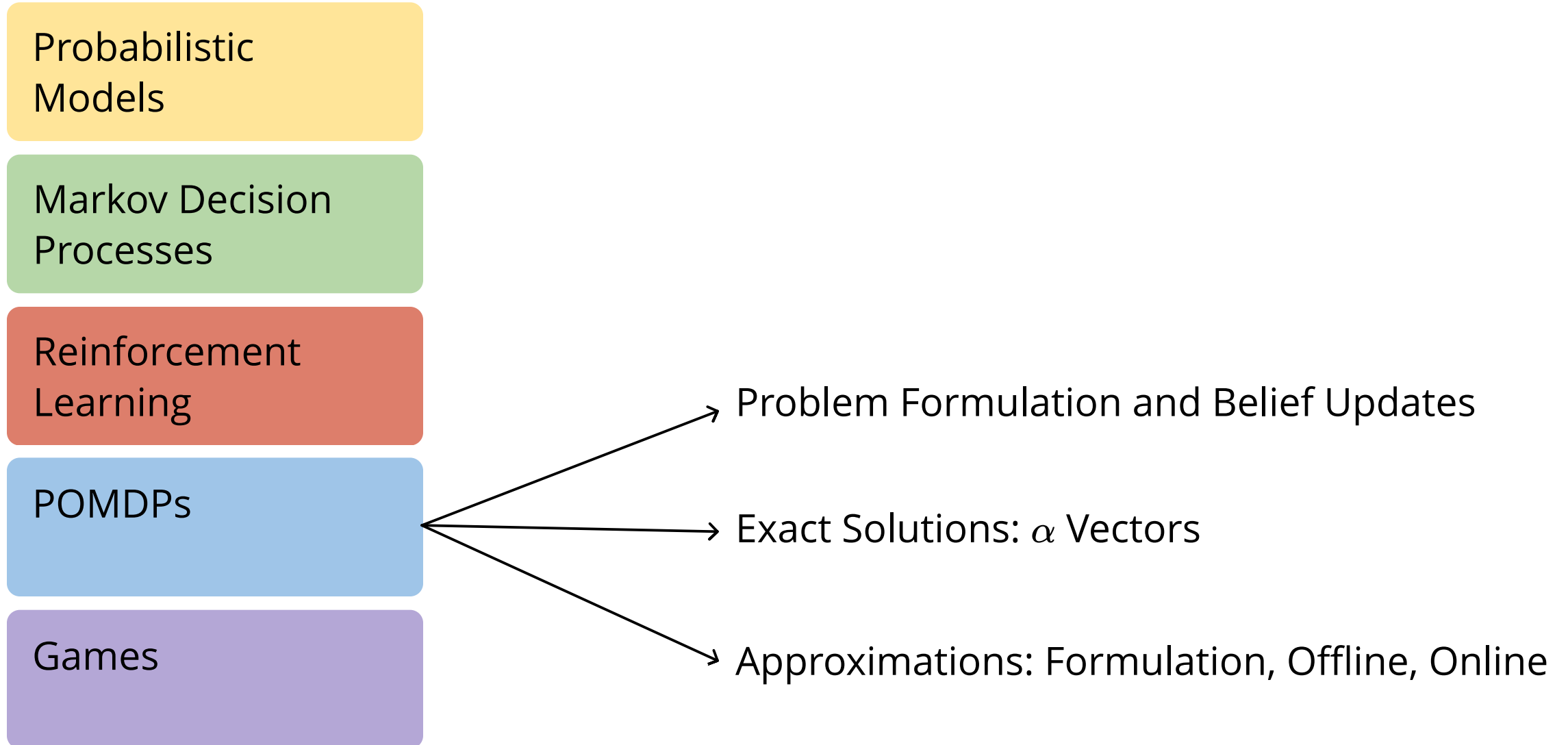
Markov Decision
Processes

Reinforcement
Learning

POMDPs

Games

DMU Recap



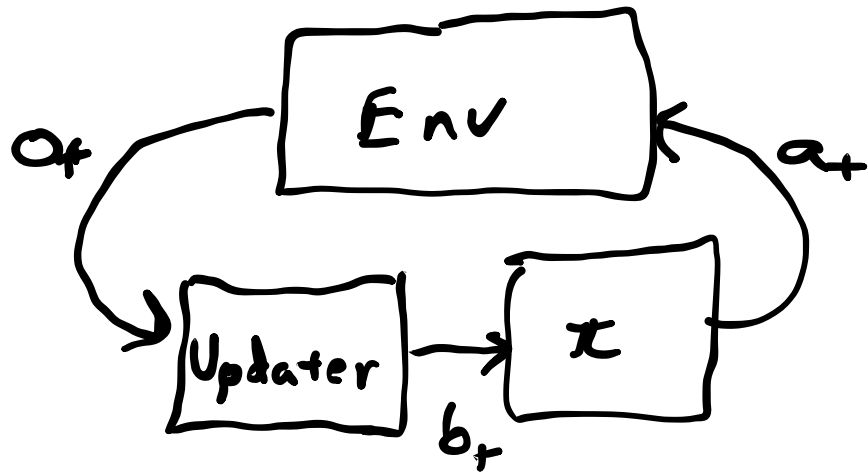
POMDPs

POMDPs

$(S, A, T, R, O, Z, \gamma)$

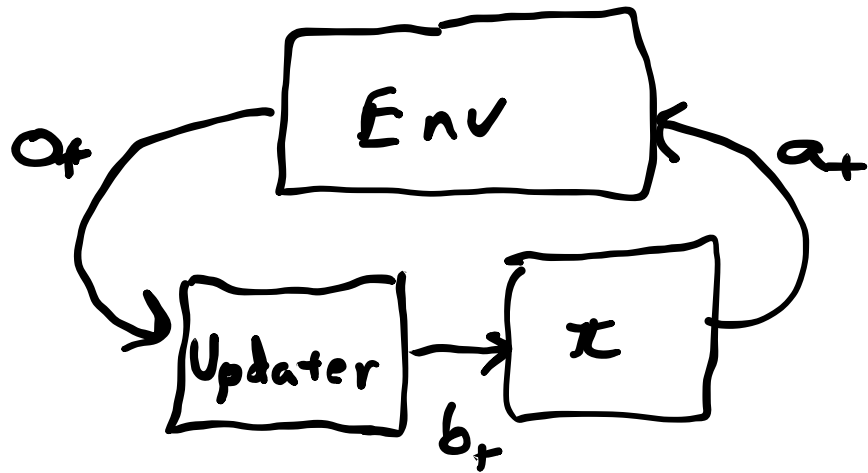
POMDPs

$(S, A, T, R, O, Z, \gamma)$



POMDPs

$(S, A, T, R, O, Z, \gamma)$

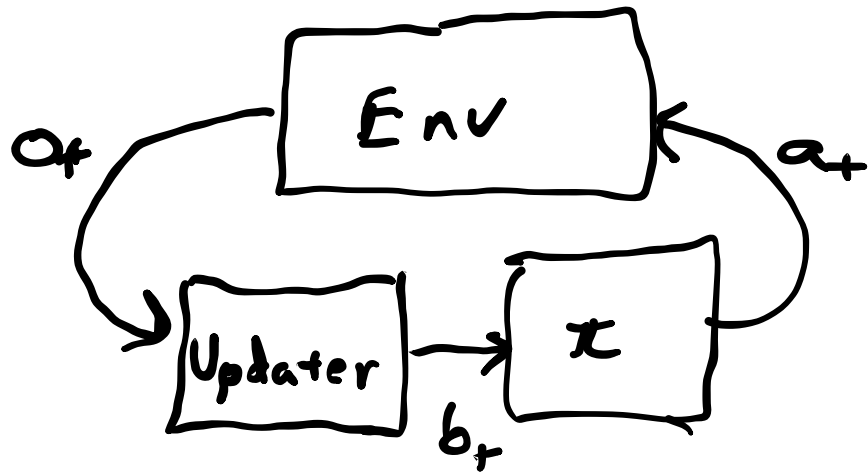


Belief Updates

- Discrete Bayesian Filter
- Particle Filter

POMDPs

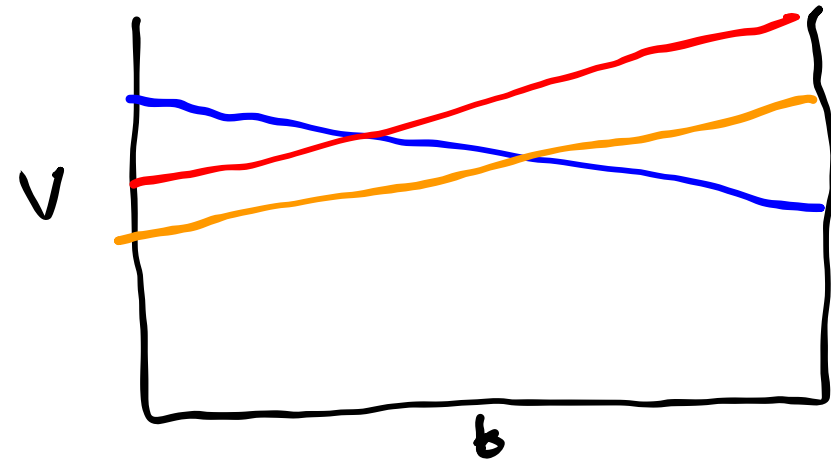
$(S, A, T, R, O, Z, \gamma)$



Belief Updates

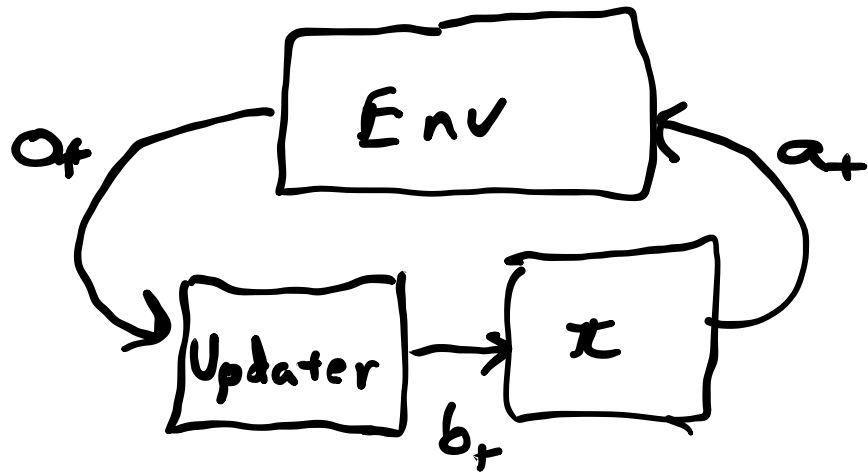
- Discrete Bayesian Filter
- Particle Filter

Alpha Vectors



POMDPs

$(S, A, T, R, O, Z, \gamma)$

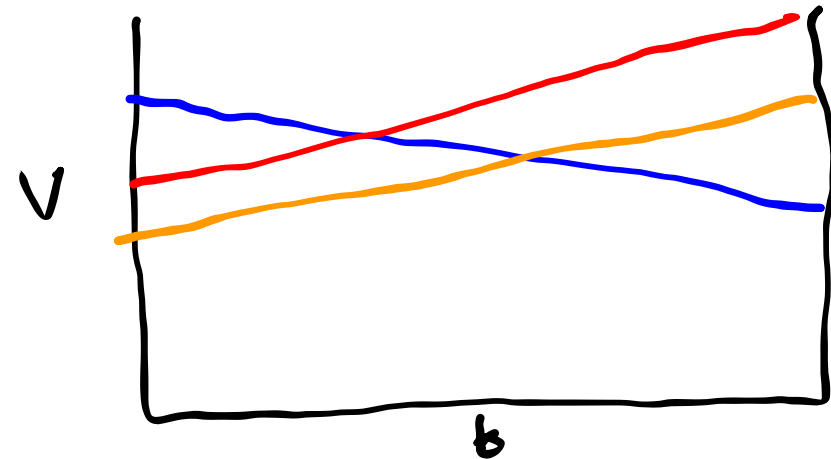


- Each alpha vector corresponds to a conditional plan

Belief Updates

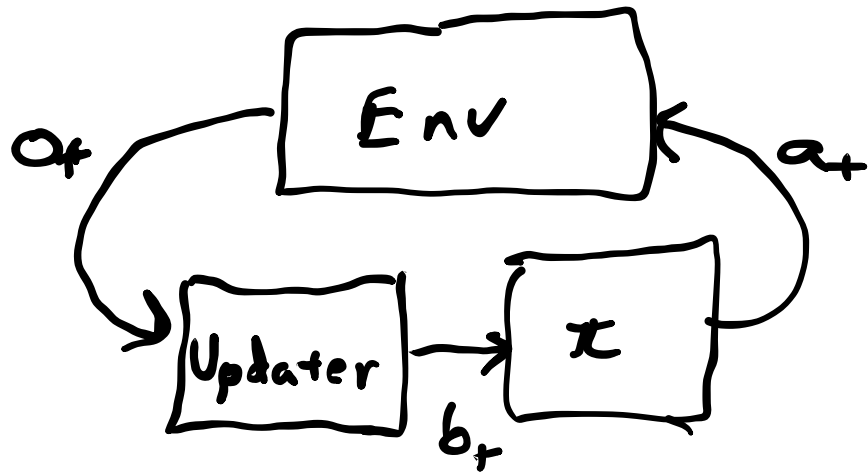
- Discrete Bayesian Filter
- Particle Filter

Alpha Vectors



POMDPs

$(S, A, T, R, O, Z, \gamma)$

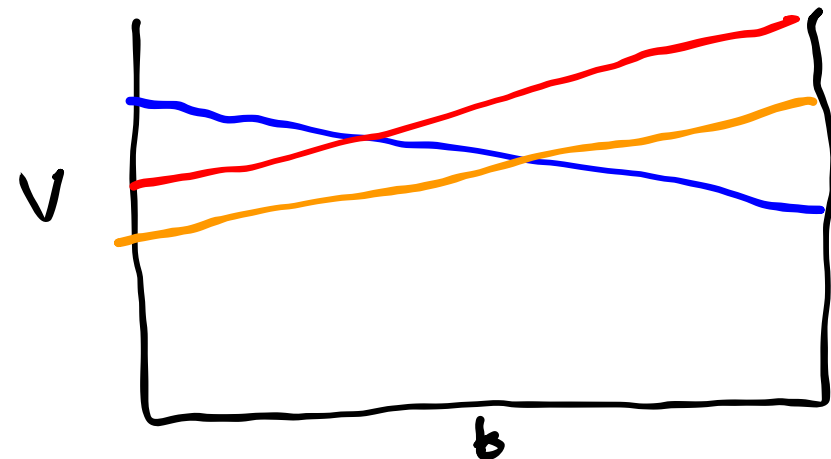


- Each alpha vector corresponds to a conditional plan
- You can prune alpha vectors by solving an LP

Belief Updates

- Discrete Bayesian Filter
- Particle Filter

Alpha Vectors



POMDP Approximations

POMDP Approximations

Formulation

- Certainty Equivalence
- QMDP
- Open Loop Feedback Control (OLFC)

POMDP Approximations

Formulation

- Certainty Equivalence
- QMDP
- Open Loop Feedback Control (OLFC)

Numerical

POMDP Approximations

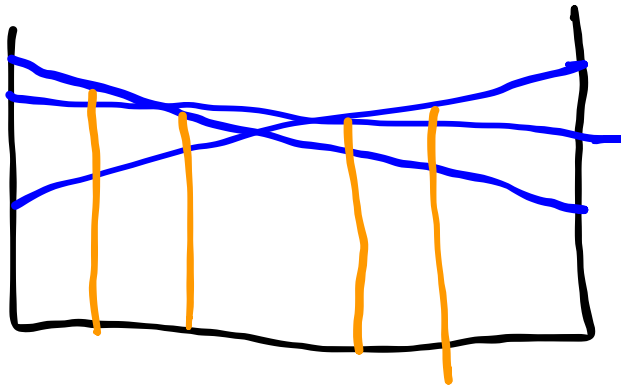
Formulation

- Certainty Equivalence
- QMDP
- Open Loop Feedback Control (OLFC)

Numerical

Offline

- Point-Based Value Iteration
- SARSOP



POMDP Approximations

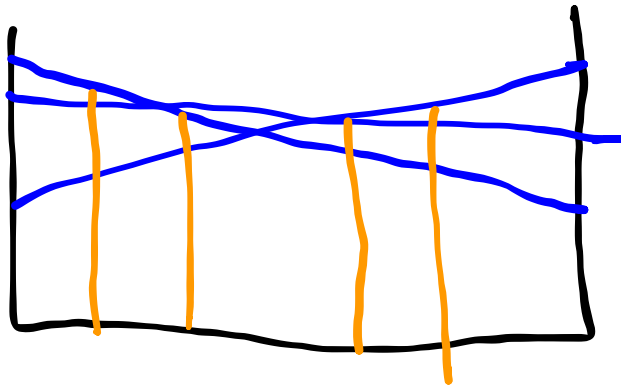
Formulation

- Certainty Equivalence
- QMDP
- Open Loop Feedback Control (OLFC)

Numerical

Offline

- Point-Based Value Iteration
- SARSOP



Online

- POMCP
- DESPOT

POMDP Approximations

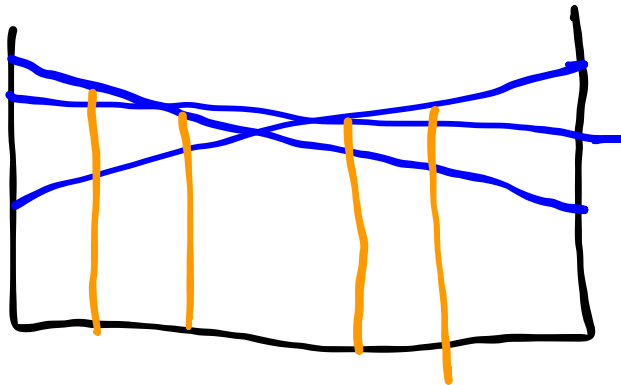
Formulation

- Certainty Equivalence
- QMDP
- Open Loop Feedback Control (OLFC)

Numerical

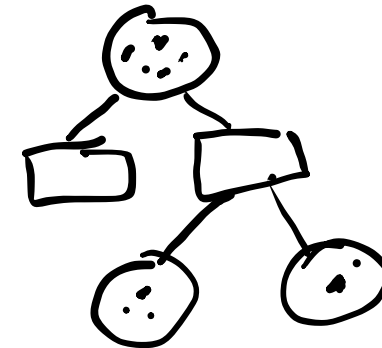
Offline

- Point-Based Value Iteration
- SARSOP



Online

- POMCP
- DESPOT



DMU Recap

Probabilistic
Models

Markov Decision
Processes

Reinforcement
Learning

POMDPs

Games

DMU Recap

Probabilistic
Models

Markov Decision
Processes

Reinforcement
Learning

POMDPs

Games

Normal Form Games and Nash Equilibria

Markov Games and POSGs

Normal Form Games

Normal Form Games

- Optimal Solutions

Normal Form Games

- ~~Optimal Solutions~~ No!

Normal Form Games

- ~~Optimal Solutions~~ No!
- Equilibria (e.g. Nash Equilibria)

Normal Form Games

- ~~Optimal Solutions~~ No!
- Equilibria (e.g. Nash Equilibria)

-1, -1	-3, 0
0, -3	-2, -2

Normal Form Games

- ~~Optimal Solutions~~ No!
- Equilibria (e.g. Nash Equilibria)

-1, -1	-3, 0
0, -3	-2, -2

- Every finite game has at least 1 Nash Equilibrium

Normal Form Games

- ~~Optimal Solutions~~ No!
- Equilibria (e.g. Nash Equilibria)

-1, -1	-3, 0
0, -3	-2, -2

- Every finite game has at least 1 Nash Equilibrium
- Might be pure or mixed

Normal Form Games

- ~~Optimal Solutions~~ No!
- Equilibria (e.g. Nash Equilibria)

-1, -1	-3, 0
0, -3	-2, -2

- Every finite game has at least 1 Nash Equilibrium
- Might be pure or mixed
- Algorithms like fictitious play converge in special cases

Markov Games and POSGS

Markov Games and POSGS

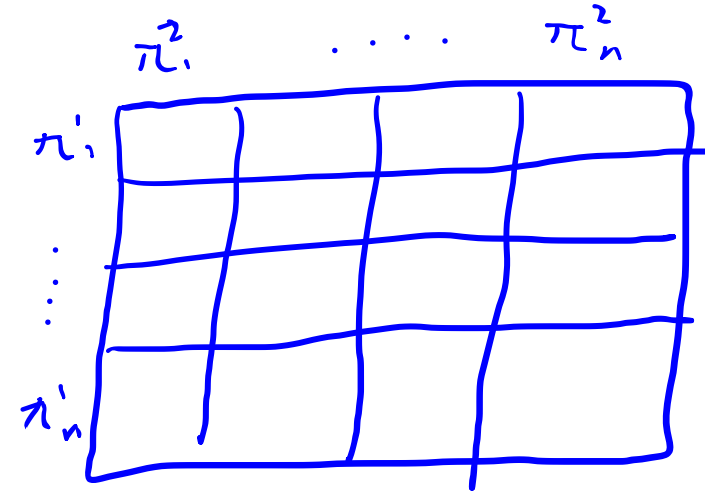
Markov Games (I, S, A, T, R, γ)

- All players play simultaneously
- Transitions are stochastic
- Best response involves solving an MDP
- Can be reduced to a simple game with policies as actions

Markov Games and POSGS

Markov Games (I, S, A, T, R, γ)

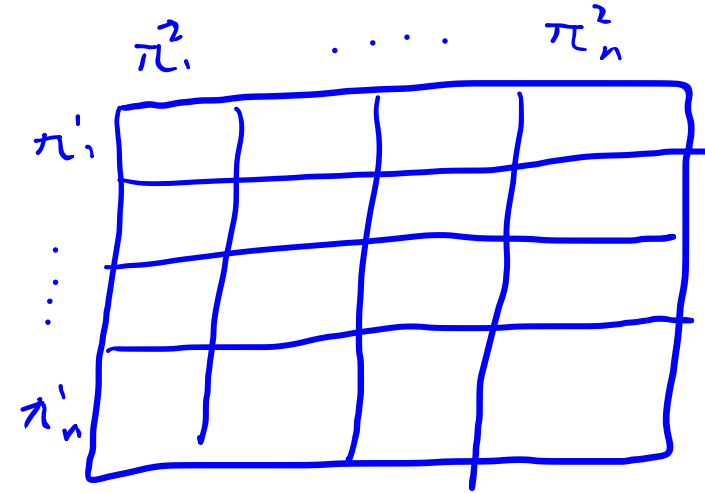
- All players play simultaneously
- Transitions are stochastic
- Best response involves solving an MDP
- Can be reduced to a simple game with policies as actions



Markov Games and POSGS

Markov Games (I, S, A, T, R, γ)

- All players play simultaneously
- Transitions are stochastic
- Best response involves solving an MDP
- Can be reduced to a simple game with policies as actions



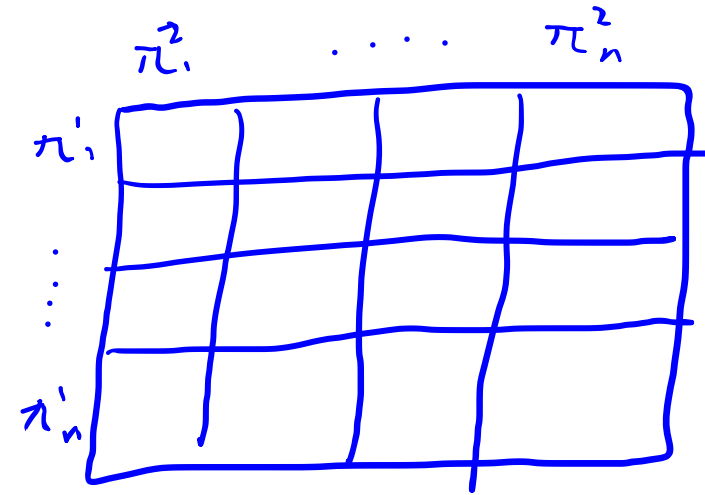
Partially Observable Markov Games

- $(I, S, A, T, R, O, Z, \gamma)$
- Each player receives a noisy observation at each step
- Beliefs not practical to compute
- Can be reduced to simple game with policies as actions

Markov Games and POSGs

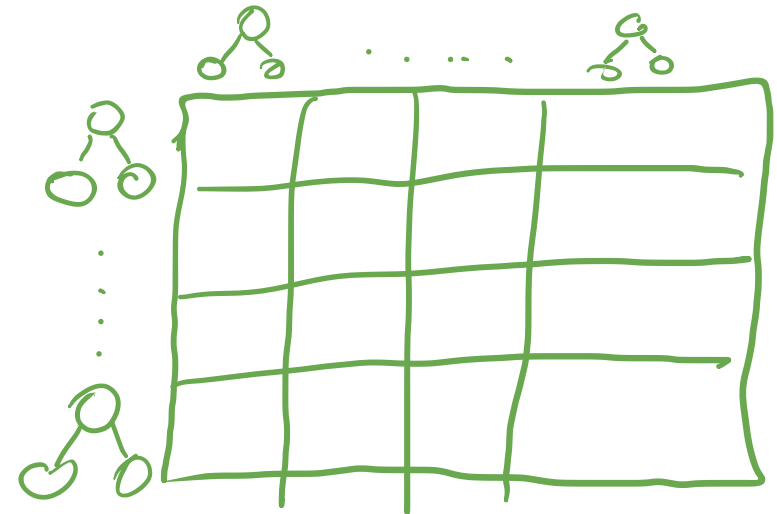
Markov Games (I, S, A, T, R, γ)

- All players play simultaneously
- Transitions are stochastic
- Best response involves solving an MDP
- Can be reduced to a simple game with policies as actions



Partially Observable Markov Games

- $(I, S, A, T, R, O, Z, \gamma)$
- Each player receives a noisy observation at each step
- Beliefs not practical to compute
- Can be reduced to simple game with policies as actions



Fictitious Play in Markov Games

Fictitious Play in Markov Games

Loop:

$$1. N(j, a^j, s) \leftarrow N(j, a^j, s) + \pi_{BR}^j(a^j \mid s) \quad \forall j, a^j, s$$

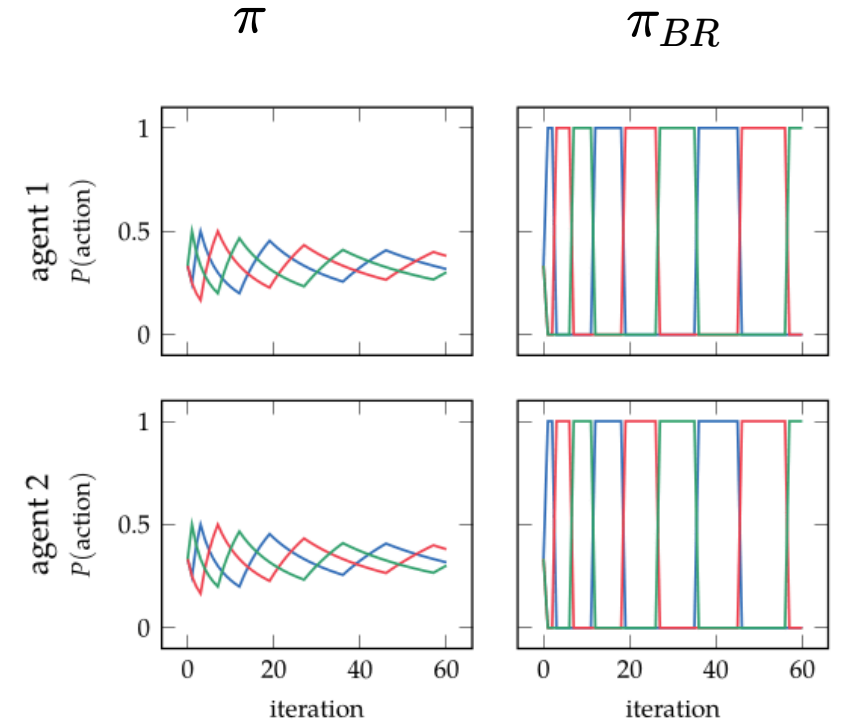
$$2. \pi^j(a^j \mid s) \propto N(j, a^j, s) \quad \forall j, a^j, s$$

$$3. \pi_{BR} \leftarrow \text{best response to } \pi$$

Fictitious Play in Markov Games

Loop:

1. $N(j, a^j, s) \leftarrow N(j, a^j, s) + \pi_{BR}^j(a^j \mid s) \quad \forall j, a^j, s$
2. $\pi^j(a^j \mid s) \propto N(j, a^j, s) \quad \forall j, a^j, s$
3. $\pi_{BR} \leftarrow$ best response to π

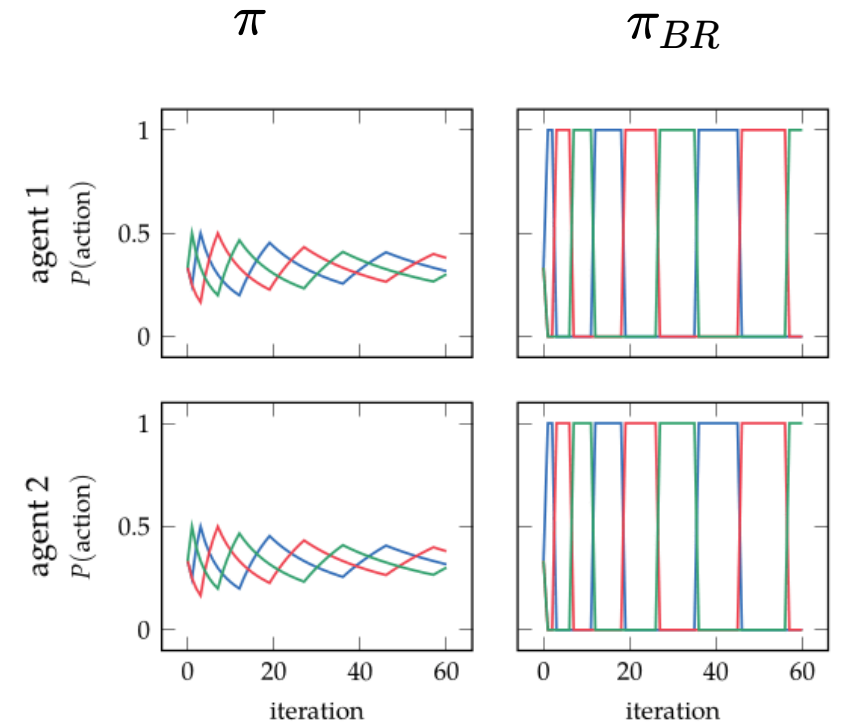


Fictitious Play in Markov Games

Loop:

1. $N(j, a^j, s) \leftarrow N(j, a^j, s) + \pi_{BR}^j(a^j \mid s) \quad \forall j, a^j, s$
2. $\pi^j(a^j \mid s) \propto N(j, a^j, s) \quad \forall j, a^j, s$
3. $\pi_{BR} \leftarrow$ best response to π

π (not necessarily π_{BR}) converges to a Nash equilibrium in many cases, notably 2-player zero-sum games



Open Questions after DMU

(Because we didn't cover them or no-one knows the answer!)

Open Questions after DMU

(Because we didn't cover them or no-one knows the answer!)

- How do we solve POSGs?

Open Questions after DMU

(Because we didn't cover them or no-one knows the answer!)

- How do we solve POSGs?
- How do we learn dynamics models to train on?

Open Questions after DMU

(Because we didn't cover them or no-one knows the answer!)

- How do we solve POSGs?
- How do we learn dynamics models to train on?
- Is imitation / supervised learning better than RL?

Open Questions after DMU

(Because we didn't cover them or no-one knows the answer!)

- How do we solve POSGs?
- How do we learn dynamics models to train on?
- Is imitation / supervised learning better than RL?
- How do we certify or build trust in these algorithms?

Open Questions after DMU

(Because we didn't cover them or no-one knows the answer!)

- How do we solve POSGs?
- How do we learn dynamics models to train on?
- Is imitation / supervised learning better than RL?
- How do we certify or build trust in these algorithms?
- Will AI replace or surpass human intelligence?

Recap

Recap

**After DMU you have basic tools to deal
with 4 Big Problems:**

Recap

**After DMU you have basic tools to deal
with 4 Big Problems:**

1. Immediate and Future Rewards

Recap

**After DMU you have basic tools to deal
with 4 Big Problems:**

1. Immediate and Future Rewards
2. Unknown Models

Recap

**After DMU you have basic tools to deal
with 4 Big Problems:**

1. Immediate and Future Rewards
2. Unknown Models
3. Partial Observability

Recap

**After DMU you have basic tools to deal
with 4 Big Problems:**

1. Immediate and Future Rewards
2. Unknown Models
3. Partial Observability
4. Other Agents